SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

Vellore Institute of Technology

REVIEW 1 | ASSESSMENT 9

MODERN AIRLINE LOGISTICS AND SUSTAINABILITY MANAGEMENT SYSTEM

A Project Report

Submitted in partial fulfilment of the requirements for the BACSE202 Lab Component Project

Submitted by

25BCE2957 | PRANALI PAYGONDA PATIL

B.Tech Computer Science and Engineering

Project Guide: Dr. Kavitha G

Head of School: Dr. Boominathan P

Academic Year 2026–27

CONTENT

CONTENT 2
LIST OF FIGURES 4
ABSTRACT 5
ACKNOWLEDGEMENT 6
1 INTRODUCTION OF THE PROJECT 7
1.1 Project overview 7
1.2 Problem statement 7
1.3 Objectives and scope 7
1.4 Scope boundaries 7
2 BACKGROUND OF THE PROJECT WORK 7
2.1 Operational and database context 7
2.2 Motivation and sustainability inspiration 8
2.3 Review 0 to Review 1 progression 8
2.4 Database platform clarification 8
3 HARDWARE AND SOFTWARE REQUIREMENTS 8
3.1 Runtime configuration 8
4 E-R DIAGRAM 9
4.1 Relationship cardinalities 9
5 DESCRIPTION ABOUT DESIGN OF THE DATABASE 10
5.1 Design approach 10
5.2 Entity summaries 10
5.3 Normalization and integrity 10
6 BACKEND DATABASE TABLES 10
7 EXECUTION OF SQL QUERIES WITH OUTPUT 11
7.1 Create and update evidence 11
7.2 Delete evidence boundary 12
7.3 Oracle behavior 12
8 FRONT END APPLICATION DESIGN 12
8.1 Screens and controls 12
8.2 Usability and data presentation 12
9 OVERALL SYSTEM DESIGN AND CONNECTIVITY 13
9.1 Front-end and back-end connectivity procedure 13
9.2 API and data flow 13
9.3 Authentication and security 13
10 PRACTICAL IMPLEMENTATION 14
10.1 Implementation stack 14

10.2 CRUD implementation 14
10.3 Demonstration workflow 14
10.4 Current implementation limits 14
11 BUSINESS ASPECT AND PRACTICAL VALUE 14
11.1 Stakeholders and value proposition 14
11.2 Business model options 14
11.3 Costs, value and adoption 15
12 SUSTAINABILITY IMPACT AND INSPIRATION 15
12.1 Potential sustainability KPIs 15
13 SCREENSHOTS OF THE APPLICATION WITH DATA 15
Figure 4. Sign-in screen for the Flight Operations workspace 16
Figure 5. Local account registration screen. 17
Figure 6. Overview cards and sample departure flow. 18
Figure 7. Flight schedule with eight records before create. 19
Figure 8. Oracle SELECT output showing initial flight rows. 20
Figure 9. Add Flight form populated for flight 309. 21
Figure 10. Oracle output after insert; nine flight rows. 22
Figure 11. Detail view for NS 309 after creation. 23
Figure 12. Edit form with schedule change and DELAYED status. 24
Figure 13. Oracle output reflecting updated flight 309. 25
Figure 14. Delete confirmation prompt for NS 301; prompt only. 26
Figure 15. Fleet module with aircraft and service details. 27
Figure 16. Crew roster with roles, bases and assignments. 28
Figure 17. Cargo tracking with category and temperature fields. 29
Figure 18. Passenger and loyalty module. 30
Figure 19. Disruptions module with causes, delay and status. 31
Figure 20. Emissions module with fuel and CO2 records. 32
Figure 21. Overview after additional flight is present. 33
14 CONCLUSION AND FUTURE SCOPE 33
14.1 Conclusion 33
14.2 Future enhancements 33
14.3 Measurable next-step plan 34
REFERENCES AND PROJECT MATERIALS 34
APPENDIX A FULL IMPLEMENTATION CODE SERVER 35
APPENDIX B FULL IMPLEMENTATION CODE FRONT END 41

LIST OF FIGURES

Figure 1. ER diagram for nine domain entities

Figure 2. Overall system architecture

Figure 3. Operational workflow

Figure 4. Sign-in page

Figure 5. Overview dashboard

Figure 6. Account registration page

Figure 7. Flight schedule

Figure 8. Oracle flight rows before insert

Figure 9. Add flight form

Figure 10. Oracle flight rows after insert

Figure 11. Flight detail

Figure 12. Edit flight form

Figure 13. Oracle updated record

Figure 14. Delete confirmation prompt

Figure 15. Fleet module

Figure 16. Crew roster module

Figure 17. Cargo tracking module

Figure 18. Passengers and loyalty module

Figure 19. Disruptions module

Figure 20. Emissions module

Figure 21. Updated overview dashboard

ABSTRACT

The Modern Airline Logistics and Sustainability Management System is a database-backed application for viewing and maintaining interconnected airline operations. It brings flight schedules, aircraft, crew, passenger loyalty, specialized cargo, emissions and disruption records into one operational workspace. The project combines the relational design documented in Review 0 with a working browser interface and a local FastAPI service connected to Oracle XE.

The data model contains nine domain entities: AIRCRAFT, FLIGHT, CREW, FLIGHT_CREW, LOYALTY_TIER, PASSENGER, SPEC_CARGO, EMISSIONS and DISRUPTION. Primary and foreign keys and integrity constraints support consistent data. The web interface provides sign-in and registration, an overview dashboard, operational modules, search and status filtering, CSV export, and create, view, update and delete actions for supported records. Passwords are stored as scrypt hashes, and authenticated sessions use signed HttpOnly cookies.

The sustainability focus is practical: fuel and CO2 records can be reviewed by flight, while specialized cargo records retain handling and temperature information. The system does not itself reduce emissions or control aircraft; it provides an information layer to support monitoring and decisions. Screenshots show the implementation with the supplied Oracle dataset, including create and update examples. The delete capture documents the confirmation step. Security, data quality, scalability and sustainability analytics remain future work.

Keywords: airline logistics, Oracle XE, FastAPI, relational database, flight operations, emissions tracking, cold-chain cargo, sustainability.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. Boominathan P, Head of the School of Computer Science and Engineering, for reviewing the project, and to my project guide, Dr. Kavitha G, for guidance throughout its development. I also thank the faculty members, classmates and others who offered feedback and support during the design, implementation and documentation of this work.

1 INTRODUCTION OF THE PROJECT

1.1 Project overview

Airline operations depend on coordinated decisions about flight schedules, aircraft availability, crew assignments, passengers, cargo and disruption response. Operational facts are related: a flight uses an aircraft, may carry several cargo shipments, can generate multiple emission logs, and may have one or more disruption events. A reliable data model must represent these links and allow staff to retrieve current records without duplicating data across disconnected lists.

This project develops a database-oriented airline operations workspace. The browser interface organizes information into distinct modules, while a local Python service reads and writes records in Oracle. Review 0 established the relational model and SQL coursework requirements; Review 1 adds application implementation and visual evidence.

1.2 Problem statement

When flight, fleet, crew, cargo and environmental information are maintained separately, staff may face inconsistent records, delayed updates and limited visibility into network status. Temperature-sensitive cargo requires explicit handling information. Emission reporting requires fuel and CO2 values to be associated with flights. Disruption records need consistent reason, delay and resolution fields. The project addresses these information-management needs through a centralized relational database and integrated interface.

1.3 Objectives and scope

Objective	Project scope
Centralize records	Linked tables and dedicated views for principal airline entities.
Support maintenance	Authenticated create, view, update and delete workflows for supported records.
Preserve consistency	Relational keys and constraints; parameterized Oracle binds in the service.
Improve inspection	Overview metrics, lists, search, status filters, refresh and CSV export.
Surface sustainability data	Store/display fuel and CO2 logs and cold-chain temperature fields.
Demonstrate DBMS concepts	Document schema design and SQL operations required by the review rubric.

1.4 Scope boundaries

The implementation is a local coursework demonstrator. It is not connected to airline reservation, airport, aircraft telemetry, payment or regulatory systems. Screenshots use demonstration records. The project does not claim real-time aircraft locations, live booking, automated crew legality checks, predictive disruption analysis or verified carbon accounting.

2 BACKGROUND OF THE PROJECT WORK

2.1 Operational and database context

Airline information is naturally relational. Flight records refer to aircraft; crew-to-flight assignments form a many-to-many relationship; passengers refer to loyalty tiers; cargo, emission logs and disruptions reference flights. A normalized schema reduces repeated descriptive information and makes referential integrity enforceable. Review 0 describes the nine-entity design and illustrates DDL, DML, query functions and joins.

2.2 Motivation and sustainability inspiration

The sustainability inspiration is to make environmental information part of everyday operations data rather than treating it as an unrelated report. Recording fuel and CO2 at flight level gives analysts a starting point for comparing routes, aircraft and operating periods. Recording required and observed cargo temperatures supports visibility for medical and perishable shipments. Better information can inform decisions, but the application itself does not prove a reduction in fuel burn or emissions.

2.3 Review 0 to Review 1 progression

Review 0 foundation	Review 1 implementation
Problem context, objectives and requirements	Local web app with registration/sign-in and modules
Nine-entity ER and relational schema	Oracle XE connectivity through FastAPI and python-oracledb
Keys, constraints and SQL coursework coverage	Browser CRUD for supported entities; search, filters, CSV export
Illustrative sample rows and query examples	Application screenshots and Oracle create/update evidence

2.4 Database platform clarification

Review 0 names MySQL 8.0 and MySQL Workbench in its software-requirements table. The delivered application code and README instead connect to Oracle XE through python-oracledb in Thin mode. This report describes the running implementation as Oracle XE. SQL syntax and key-generation behavior must follow the Oracle schema actually installed; the MySQL entry is treated as an initial coursework requirement, not the application runtime.

3 HARDWARE AND SOFTWARE REQUIREMENTS

Category	Minimum / current setup	Recommended extension
Processor	Intel Core i3 equivalent	Intel Core i5 or higher
Memory	4 GB RAM	8 GB RAM or higher
Storage	20 GB free	SSD for database, backups and logs
Operating system	Windows 10/11, Linux or macOS	Supported 64-bit OS with updates
Database	Oracle XE schema used by app	Managed Oracle with backup/monitoring
Backend	Python 3.10+; FastAPI; python-oracledb	Pinned dependencies; production ASGI
Frontend	Modern browser; HTML/CSS/JavaScript	Current Chrome, Edge or Firefox
Network	Local TCP access to Oracle	TLS-protected private network

3.1 Runtime configuration

The README instructs the operator to install requirements.txt dependencies, configure Oracle credentials and service DSN in a private .env file, and start the application on 127.0.0.1:8001. The browser should use the FastAPI-served page rather than opening index.html directly. The configured service must point to the same Oracle container/schema as the airline tables. Credentials and the session secret must remain private and must not be included in a report or shared repository.

4 E-R DIAGRAM

This ER model reproduces the nine domain entities documented in Review 0. FLIGHT is the operational hub. FLIGHT_CREW resolves the many-to-many relationship between flights and crew. Foreign keys connect related records. The authentication support table is separate from the nine-entity domain model.

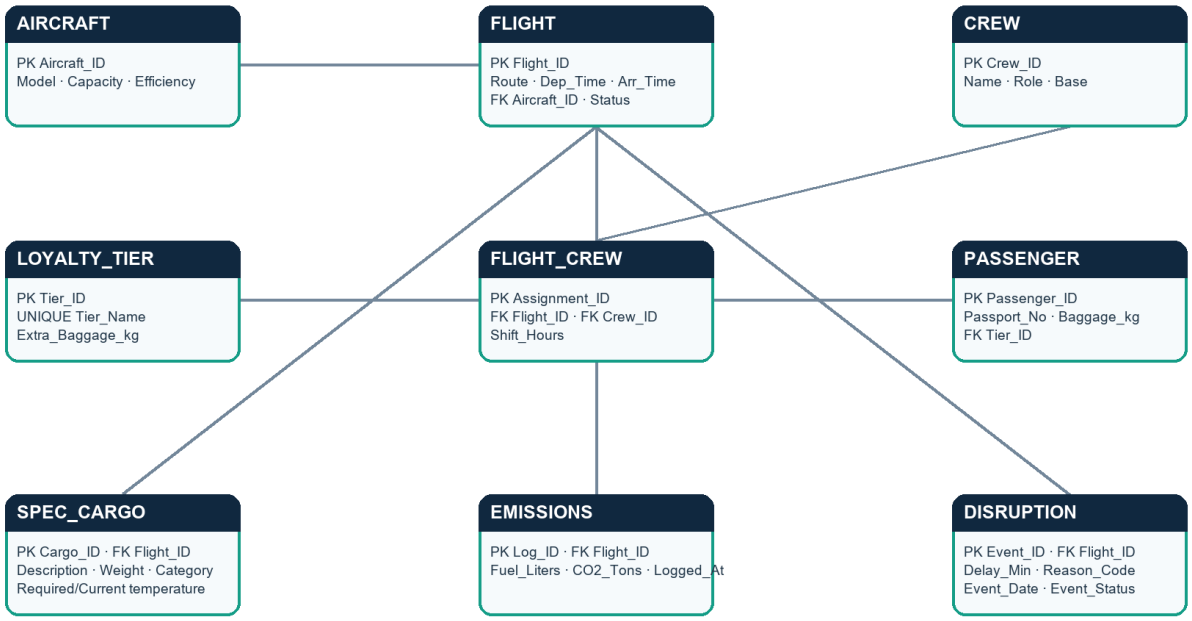


Figure 1. ER diagram for the nine domain entities; PK/FK labels identify keys.

4.1 Relationship cardinalities

Parent	Relationship	Child	Cardinality / meaning
AIRCRAFT	assigned to	FLIGHT	1:M — each flight uses one aircraft; aircraft can operate flights over time.
FLIGHT	assigned through	FLIGHT_CREW	1:M — a flight can have multiple crew assignments.
CREW	assigned through	FLIGHT_CREW	1:M — a crew member can have multiple assignment records.
LOYALTY_TIER	assigned to	PASSENGER	1:M — each passenger has a tier; a tier can include many passengers.
FLIGHT	carries	SPEC_CARGO	1:M — a flight can carry multiple shipments.
FLIGHT	generates	EMISSIONS	1:M — a flight can have multiple emission logs.
FLIGHT	experiences	DISRUPTION	1:M — a flight can have multiple disruption events.

5 DESCRIPTION ABOUT DESIGN OF THE DATABASE

5.1 Design approach

The design separates entities by responsibility and uses primary keys to identify records. Foreign keys express operational relationships. Unique constraints are appropriate for tier names and passport numbers. NOT NULL constraints protect required fields, CHECK constraints constrain valid ranges or enumerated statuses, and DEFAULT constraints supply initial values where appropriate. The database remains the final authority for referential integrity during application writes.

5.2 Entity summaries

Entity	Purpose	Important links / integrity
AIRCRAFT	Fleet model, capacity, efficiency, service status	PK Aircraft_ID; referenced by FLIGHT.
FLIGHT	Route, departure/arrival, aircraft and status	PK Flight_ID; FK Aircraft_ID.
CREW	Crew identity, role and base	PK Crew_ID; linked by FLIGHT_CREW.
FLIGHT_CREW	Assignment bridge and shift hours	FK Flight_ID + FK Crew_ID; resolves M:N.
LOYALTY_TIER	Tier label and extra baggage allowance	PK Tier_ID; Tier_Name unique.
PASSENGER	Passenger, passport, tier and baggage	Passport_No unique; FK Tier_ID.
SPEC_CARGO	Shipment category, weight and temperatures	FK Flight_ID; status and temperature fields.
EMISSIONS	Fuel and CO2 by timestamp	FK Flight_ID; route/flight analysis.
DISRUPTION	Delay, reason, date and status	FK Flight_ID; event tracking.

5.3 Normalization and integrity

Separating aircraft, flights, crew, tiers, passengers and flight-linked records avoids storing repeated descriptive attributes in the flight table. FLIGHT_CREW prevents multi-valued crew lists in a single field. Bound values in the API reduce SQL injection risk for values, while database constraints reject invalid relationships and domains.

6 BACKEND DATABASE TABLES

This inventory presents the Review 0 attributes and logical Oracle-friendly types. Exact deployed widths should be verified against Oracle metadata before a production migration; undocumented lengths are not inferred here. Key and uniqueness requirements follow the project schema.

Table	PK	Type	Other attributes (logical types)	Constraints / links
AIRCRAFT	AIRCRAFT_ID	NUMBER	MODEL VARCHAR2; CAPACITY_KG NUMBER; FUEL_EFFICIENCY NUMBER; AIRCRAFT_STATUS VARCHAR2	PK; valid capacity/status
FLIGHT	FLIGHT_ID	NUMBER	ROUTE_CODE VARCHAR2; DEP_TIME TIMESTAMP; ARR_TIME TIMESTAMP; AIRCRAFT_ID NUMBER; FLIGHT_STATUS VARCHAR2	PK; FK AIRCRAFT_ID; required fields
CREW	CREW_ID	NUMBER	CREW_NAME VARCHAR2; ROLE VARCHAR2; BASE_LOCATION VARCHAR2	PK; required identifying fields
FLIGHT_CREW	ASSIGNMENT_ID	NUMBER	FLIGHT_ID NUMBER; CREW_ID NUMBER; SHIFT_HOURS NUMBER	PK; two FKs; valid hours

LOYALTY_TIER	TIER_ID	NUMBER	TIER_NAME VARCHAR2; EXTRA_BAGGAGE_KG NUMBER	PK; unique name; nonnegative allowance
PASSENGER	PASSENGER_ID	NUMBER	PASSENGER_NAME VARCHAR2; PASSPORT_NO VARCHAR2; TIER_ID NUMBER; BAGGAGE_KG NUMBER	PK; unique passport; FK tier
SPEC_CARGO	CARGO_ID	NUMBER	FLIGHT_ID NUMBER; DESCRIPTION VARCHAR2; WEIGHT_KG NUMBER; CATEGORY VARCHAR2; REQ_TEMP_C NUMBER; CURRENT_TEMP_C NUMBER; TRACKING_STATUS VARCHAR2	PK; FK flight; nonnegative weight
EMISSIONS	LOG_ID	NUMBER	FLIGHT_ID NUMBER; FUEL_LITERS NUMBER; CO2_TONS NUMBER; LOGGED_AT TIMESTAMP	PK; FK flight; nonnegative values
DISRUPTION	EVENT_ID	NUMBER	FLIGHT_ID NUMBER; DELAY_MIN NUMBER; REASON_CODE VARCHAR2; EVENT_DATE TIMESTAMP; EVENT_STATUS VARCHAR2	PK; FK flight; nonnegative delay

Implementation note: the FastAPI service also creates APP_LOGIN_USERS as needed. It contains identity-generated USER_ID, NAME, EMAIL and PASSWORD_HASH for authentication, separate from the nine airline-domain entities.

7 EXECUTION OF SQL QUERIES WITH OUTPUT

Review 0 contains the coursework SQL section: table creation with constraints, five-row inserts, delete and update demonstrations for the domain tables, SELECT, single-row functions, aggregate functions, GROUP BY, HAVING, subqueries and joins. The application screenshots add Oracle terminal evidence for flight records and the live create/update path. The application issues parameterized Oracle SELECT, INSERT, UPDATE and DELETE statements through python-oracledb.

Required operation	Evidence / purpose
CREATE TABLE + constraints	Review 0 DDL; PK/FK and domain integrity establish schema.
INSERT up to five sample rows	Review 0 sample rows; app insert shown for flight ID 309.
DELETE	Review 0 deletion examples; UI confirmation prompt shown.
UPDATE	Review 0 per-table examples; flight 309 change shown in app and query output.
SELECT	Review 0 examples and live Oracle query outputs.
Single-row functions	Review 0 examples for text, date and numeric transformations.
Aggregate functions	Review 0 examples for counts, totals and averages.
GROUP BY / HAVING	Review 0 grouped summaries and filtered groups.
Subqueries	Review 0 nested query examples.
JOIN	Review 0 joins across aircraft/flights, passengers/tiers, crew and flight assignments, cargo and disruptions.

7.1 Create and update evidence

The supplied captures show eight initial flight rows, an Add Flight form for ID 309, and query output after insertion with nine rows. A detail view shows NS 309. The edit form changes its schedule and status to DELAYED; the subsequent Oracle output reflects that updated record. For this example, the evidence links the UI action to the database result.

7.2 Delete evidence boundary

The delete screenshot shows the browser confirmation prompt for NS 301. It evidences the confirmation step and intended delete action. The supplied capture set does not show a post-confirmation list or query proving absence, so this report does not claim the deletion result is independently verified by that screenshot.

7.3 Oracle behavior

The live implementation uses Oracle. The service binds values and commits successful writes. Numeric key generation depends on deployed identity/default/trigger behavior; the README flags this as a setup check. Confirm key generation and actual constraints in Oracle metadata before extending the system.

8 FRONT END APPLICATION DESIGN

The browser interface in index.html uses a persistent sidebar and content area. Its theme combines dark navigation, a light work surface, summary cards, record tables and modal forms. The supplied screens establish the Northstar Airways / Flight Operations identity for the demonstration.

8.1 Screens and controls

Area	Visible purpose / controls
Authentication	Sign-in and account creation before operational data.
Overview	Flight count, active fleet, open disruptions, CO2, departure visualization and upcoming flights.
Flight schedule	Route/time/aircraft/status; add, detail, edit, delete, search, filter, CSV export.
Fleet	Aircraft, capacity, fuel efficiency, today's flight and status.
Crew roster	Name, role, base, assignment, hours and duty status.
Cargo tracking	Shipment, category, flight, weight, temperature and tracking status.
Passengers & loyalty	Passenger, tier, baggage and allowance.
Disruptions	Flight, route, reason, delay, event timestamp and status.
Emissions	Flight fuel/CO2 logs, summary metrics and timestamps.

8.2 Usability and data presentation

Search and status filters narrow lists; CSV export supports offline review; refresh reloads current records. Overview cards provide a summary, while module tables show record-level details. The departure chart is labelled as sample data and is illustrative unless the underlying aggregation is verified. Counts are screenshots of a changing demo dataset, not a live airline network report.

9 OVERALL SYSTEM DESIGN AND CONNECTIVITY

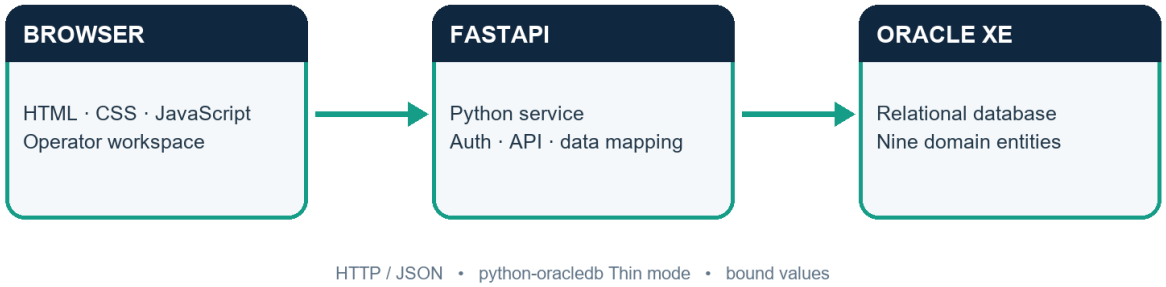


Figure 2. Browser, FastAPI service and Oracle XE architecture.

9.1 Front-end and back-end connectivity procedure

Step	Procedure
1. Configure	Keep Oracle credentials, service DSN and session secret in private environment settings.
2. Start service	Launch FastAPI at 127.0.0.1:8001; serve index.html from the same origin.
3. Authenticate	Register/sign in; server verifies the hash and issues a signed HttpOnly session cookie.
4. Read	Browser requests API data; service uses a pooled python-oracledb connection.
5. Write	Create/edit/delete calls map to allowed tables, bind values and commit successful writes.
6. Refresh	Browser reloads current Oracle records; README indicates refresh every five seconds.

9.2 API and data flow

The service provides health and authentication endpoints, consolidated data, reference lookups and entity-level create/update/delete routes. It maps presentation fields to Oracle columns and derives some view values from related tables. Health reports database connectivity and table visibility. Operational reads and writes require an authenticated user.

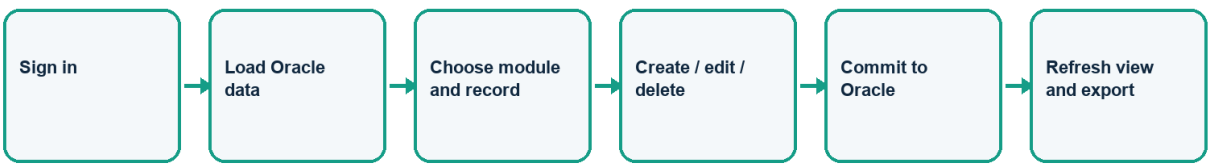


Figure 3. Typical authenticated data-maintenance workflow.

9.3 Authentication and security

The README and source describe scrypt password hashing, signed HttpOnly session cookies with a 12-hour expiry, a private session secret, and APP_LOGIN_USERS rather than the existing subscription-oriented USERS table. All

accounts currently share the same permissions; role-based access is not implemented. The app is intended for local coursework and should not be exposed to an untrusted network without production security work.

10 PRACTICAL IMPLEMENTATION

10.1 Implementation stack

Layer	Technology / role
Browser	HTML, CSS and JavaScript for screens and interaction.
Application service	Python FastAPI for authentication, API routing, mapping and validation.
Database adapter	python-oracledb Thin mode and connection pooling.
Database	Oracle XE tables, relationships and constraints.
Transport	Local HTTP in coursework; JSON request/response bodies.

10.2 CRUD implementation

The API centralizes entity mapping so write logic can build INSERT or UPDATE statements from permitted fields and bind values. Delete routes target record keys and commit the operation. The interface presents forms and detail modals and reloads records after changes. Foreign-key violations, duplicates, invalid types and missing key generation remain database errors that should be clearly communicated.

10.3 Demonstration workflow

The screenshots cover sign-in and account creation; overview; flight list and detail; creation of NS 309; the Oracle row count change from eight to nine; an edit changing status to DELAYED; Oracle output reflecting the update; delete confirmation; and the fleet, crew, cargo, passenger, disruptions and emissions modules.

10.4 Current implementation limits

The service is local-only, has no separate user roles and uses illustrative sample views in parts of the overview. The evidence does not establish equivalent write testing for every module. FLIGHT_CREW is in the schema, while the visible menu has a crew roster and no dedicated assignment-management screen. These are practical next targets.

11 BUSINESS ASPECT AND PRACTICAL VALUE

11.1 Stakeholders and value proposition

Potential users include operations coordinators, fleet planners, cargo teams and sustainability analysts. A consolidated operational view can reduce time spent switching between lists, make update ownership clearer and make sustainability fields available alongside flights. The demonstrator illustrates this concept; operational savings have not been measured.

11.2 Business model options

Model	Application	Considerations
Internal deployment	Operate as an internal support workspace.	Needs integration, access control, audit, uptime and governance.

SaaS subscription	Tier by airline size, users or modules.	Needs tenant isolation, cloud hosting, support and compliance.
Modular add-ons	Cargo temperature, disruption analytics or emissions reports.	Needs verified data definitions and reliable integrations.
Implementation services	Configuration, migration and staff training.	Requires domain validation and maintenance.

11.3 Costs, value and adoption

A commercial estimate needs discovery rather than speculative pricing. Cost drivers include hosting, database licensing, identity management, monitoring, backups, security review, integration, support and training. Potential value drivers include less manual reconciliation, faster exception visibility, better cargo traceability and easier fuel/emissions reporting. A pilot should measure task time, data error rate, disruption response time and emissions-record completeness against a baseline.

A practical adoption path is to validate the model with an operations stakeholder, pilot a limited route or team, connect approved source systems, reconcile in parallel, train users and expand only after data-quality and security gates pass. This student build is a proof of concept, not production-ready software.

12 SUSTAINABILITY IMPACT AND INSPIRATION

The project was inspired by the need to make sustainability measurable in operational workflows. Fuel and CO2 logging makes environmental impact visible at flight level, and fuel-efficiency fields create a basis for comparison. Cargo temperature fields support careful movement of medical and perishable goods and can help reduce spoilage when connected to reliable measurements and response processes.

Environmental benefit depends on accurate data and decisions. The application stores and presents records; it does not independently measure emissions, calculate lifecycle impact, optimize routes or prove carbon reduction. Future work should document emission factors, provenance, reporting boundaries and verification before external sustainability claims.

12.1 Potential sustainability KPIs

KPI	Example definition	Data needed
Fuel per flight	Fuel liters per flight, normalized by distance	Fuel log; route distance; aircraft type
CO2 intensity	CO2 per passenger-km or tonne-km	Verified emissions; loads; distance
Cold-chain excursion rate	Shipments outside allowed range / monitored shipments	Timestamped sensor readings and thresholds
Data completeness	Flights with required logs / operated flights	Schedule and emission logs

13 SCREENSHOTS OF THE APPLICATION WITH DATA

Figures below reproduce the supplied application evidence. Counts, dates and values are shown as captured; dashboard metrics and sample records are illustrative, not live airline reports.

Demo video (Google Drive): link not provided. Insert the shareable demonstration URL here before submission.

Figure 4. Sign-in screen for the Flight Operations workspace.

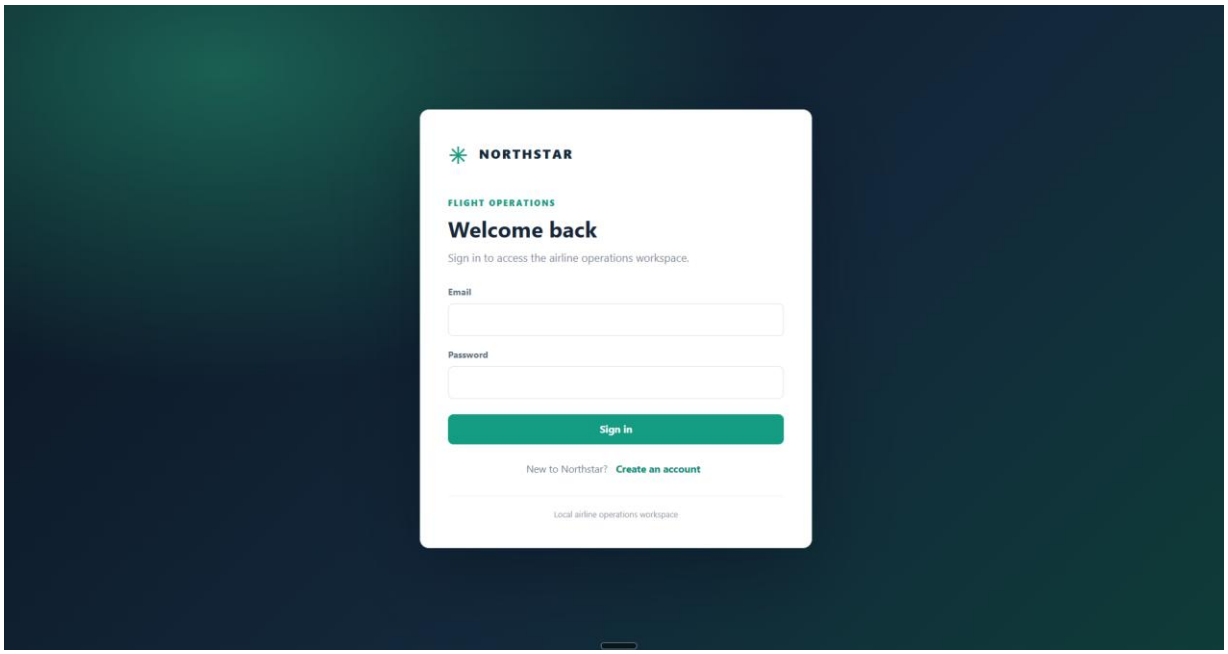


Figure 4. Sign-in screen for the Flight Operations workspace.

Figure 5. Local account registration screen.

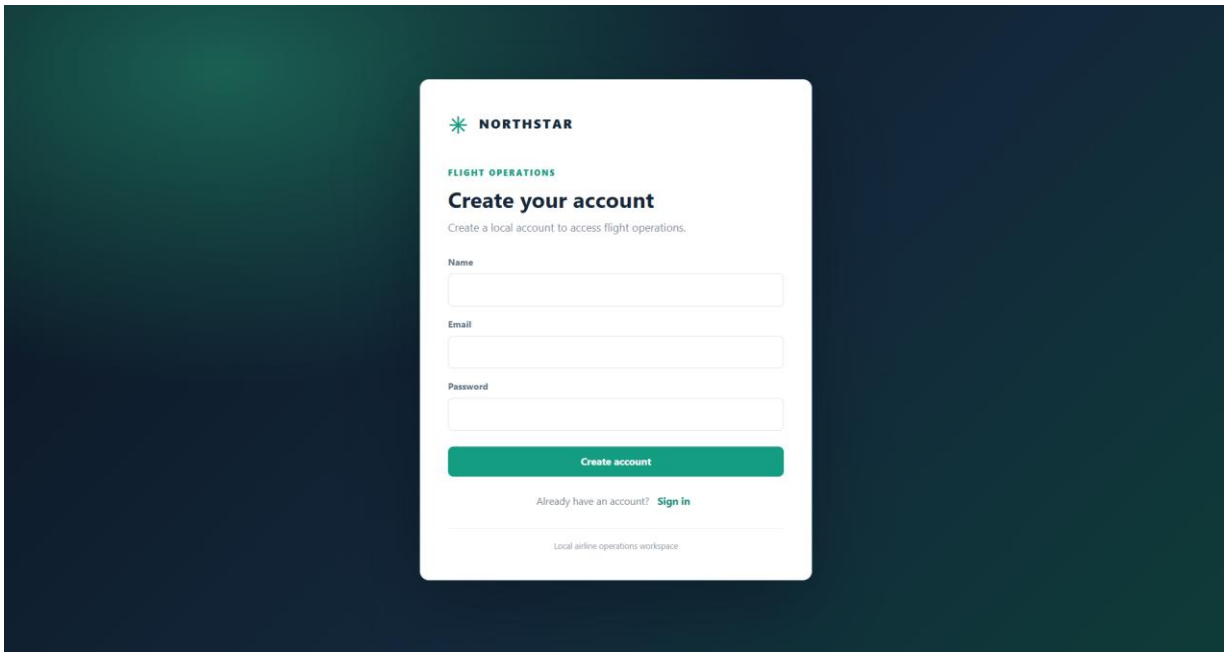


Figure 5. Local account registration screen.

Figure 6. Overview cards and sample departure flow.

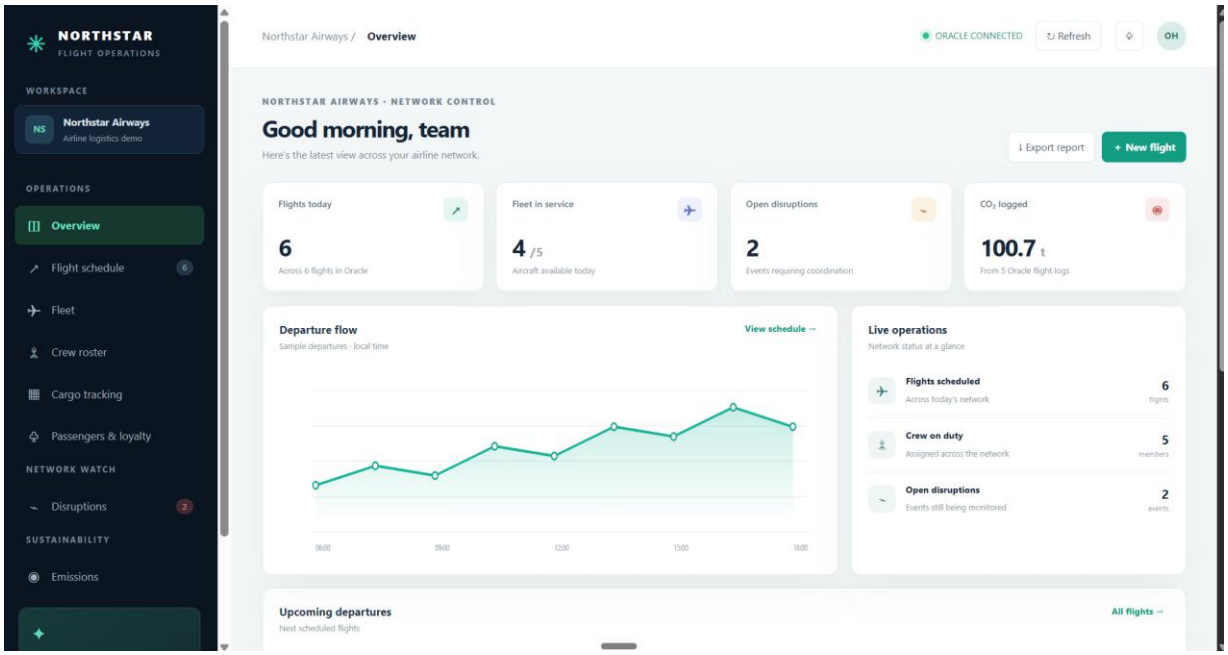


Figure 6. Overview cards and sample departure flow.

Figure 7. Flight schedule with eight records before create.

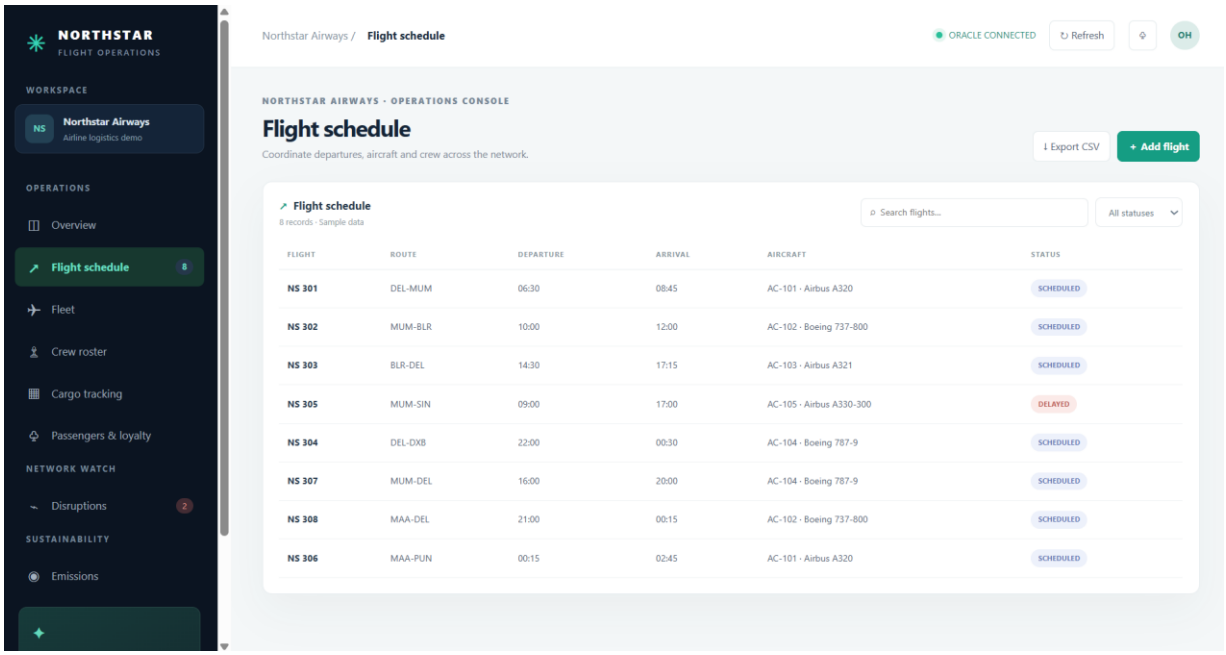


Figure 7. Flight schedule with eight records before create.

Figure 8. Oracle SELECT output showing initial flight rows.



```
SQL> select * from flight;

 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       301 DEL-MUM
20-AUG-26 06.30.00.000000 AM
20-AUG-26 08.45.00.000000 AM
       101 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       302 MUM-BLR
20-AUG-26 10.00.00.000000 AM
20-AUG-26 12.00.00.000000 PM
       102 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       303 BLR-DEL
20-AUG-26 02.30.00.000000 PM
20-AUG-26 05.15.00.000000 PM
       103 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       304 DEL-DXB
21-AUG-26 10.00.00.000000 PM
22-AUG-26 12.30.00.000000 AM
       104 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       305 MUM-SIN
21-AUG-26 09.00.00.000000 AM
21-AUG-26 05.00.00.000000 PM
       105 DELAYED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       307 MUM-DEL
10-OCT-26 04.00.00.000000 PM
10-OCT-26 08.00.00.000000 PM
       104 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       306 MAA-PUN
30-OCT-26 12.15.00.000000 AM
31-OCT-26 02.45.00.000000 AM
       101 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       308 MAA-DEL
13-OCT-26 09.00.00.000000 PM
14-OCT-26 12.15.00.000000 AM
       102 SCHEDULED


8 rows selected.
```

Figure 8. Oracle SELECT output showing initial flight rows.

Figure 9. Add Flight form populated for flight 309.

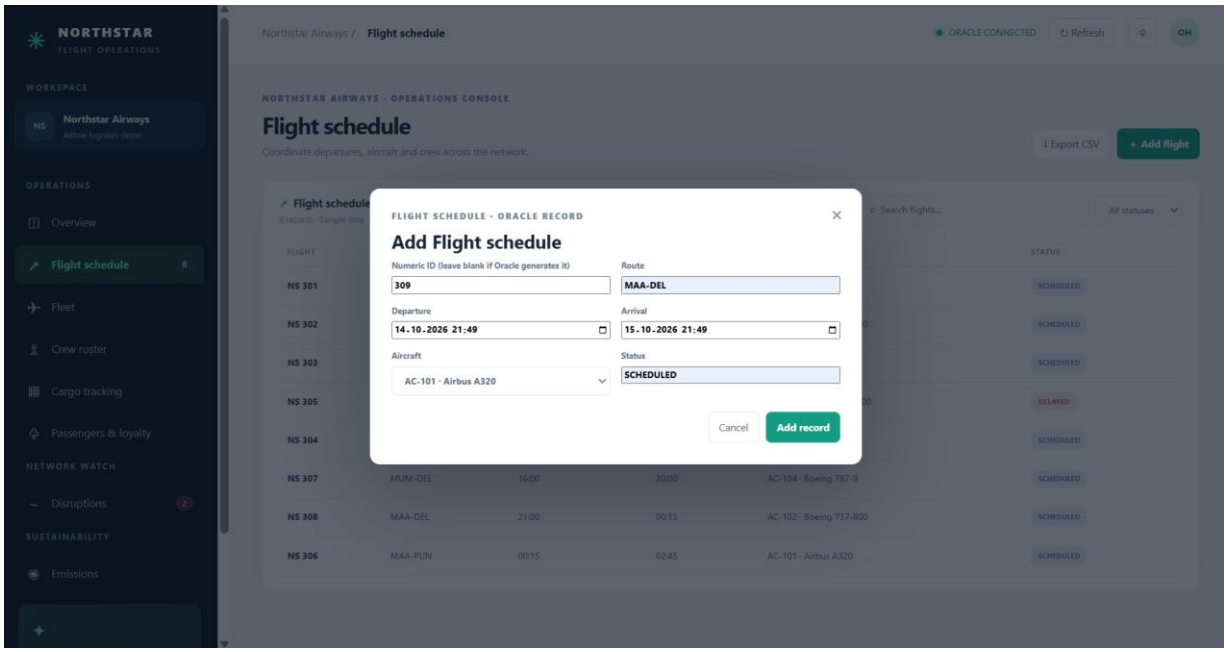


Figure 9. Add Flight form populated for flight 309.

Figure 10. Oracle output after insert; nine flight rows.



```
SQL> select * from flight;

 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
        301 DEL-MUM
20-AUG-26 06.30.00.000000 AM
20-AUG-26 08.45.00.000000 AM
        101 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
        302 MUM-BLR
20-AUG-26 10.00.00.000000 AM
20-AUG-26 12.00.00.000000 PM
        102 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
        303 BLR-DEL
20-AUG-26 02.30.00.000000 PM
20-AUG-26 05.15.00.000000 PM
        103 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
        304 DEL-DXB
21-AUG-26 10.00.00.000000 PM
22-AUG-26 12.30.00.000000 AM
        104 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
        305 MUM-SIN
21-AUG-26 09.00.00.000000 AM
21-AUG-26 05.00.00.000000 PM
        105 DELAYED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
        307 MUM-DEL
10-OCT-26 04.00.00.000000 PM
10-OCT-26 08.00.00.000000 PM
        104 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
        306 MAA-PUN
30-OCT-26 12.15.00.000000 AM
31-OCT-26 02.45.00.000000 AM
        101 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
        308 MAA-DEL
13-OCT-26 09.00.00.000000 PM
14-OCT-26 12.15.00.000000 AM
        102 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
        309 MAA-DEL
14-OCT-26 09.49.00.000000 PM
15-OCT-26 09.49.00.000000 PM
        101 SCHEDULED

9 rows selected.
```

Figure 10. Oracle output after insert; nine flight rows.

Figure 11. Detail view for NS 309 after creation.

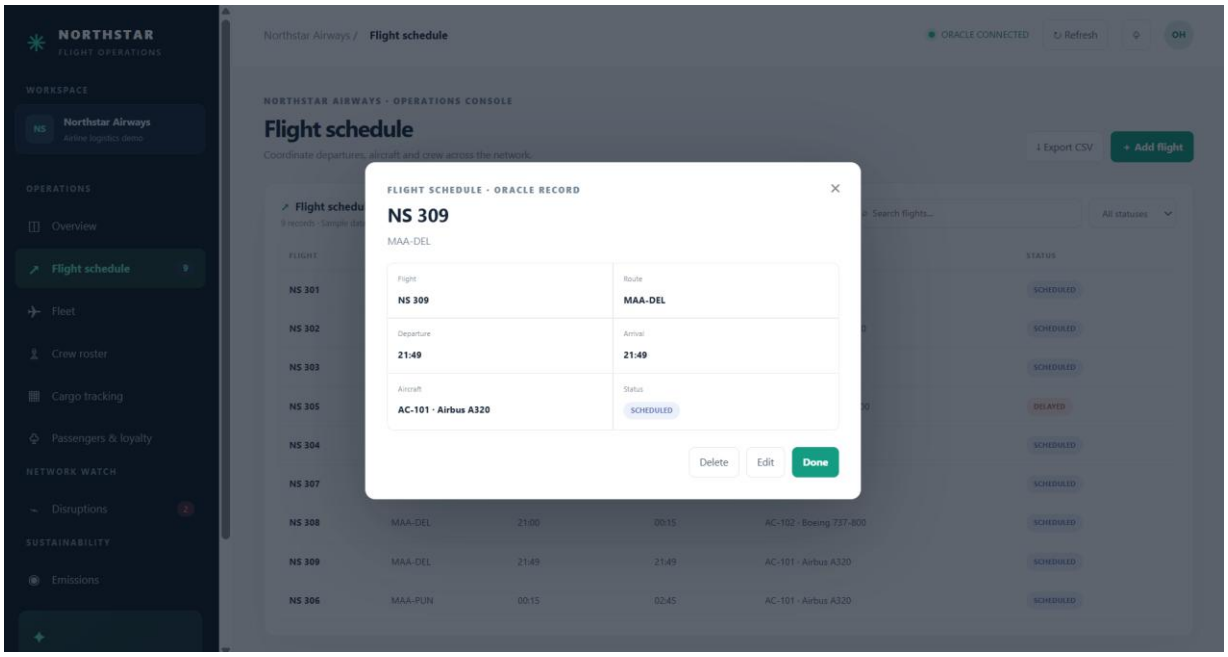


Figure 11. Detail view for NS 309 after creation.

Figure 12. Edit form with schedule change and DELAYED status.

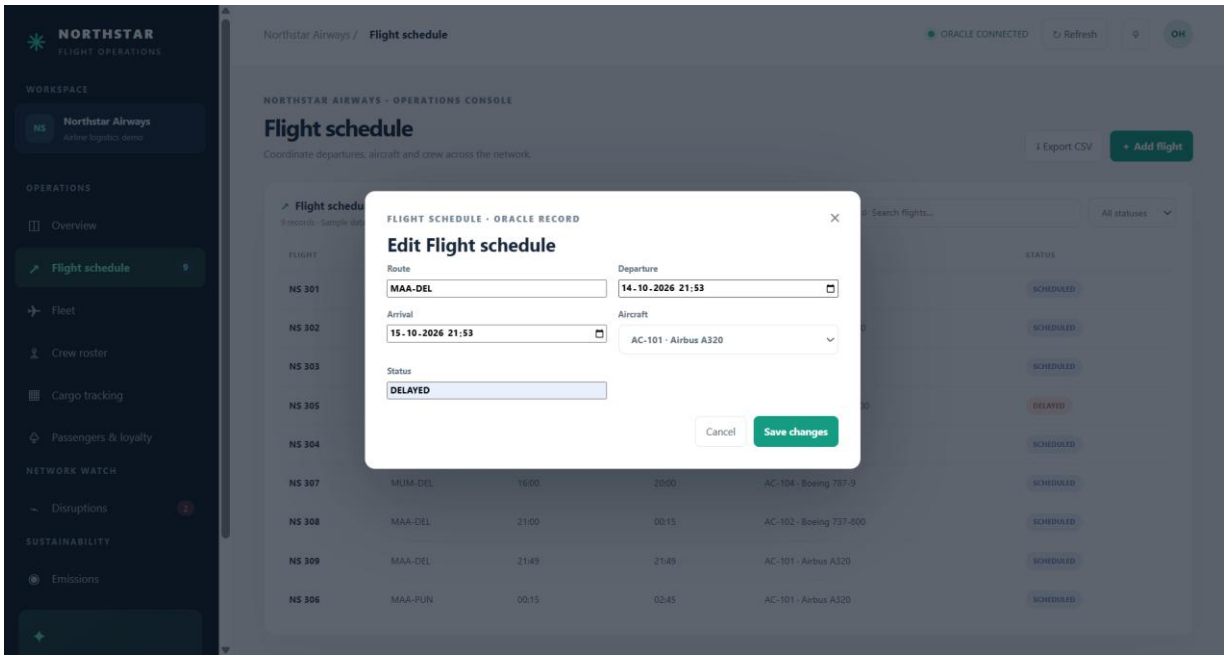


Figure 12. Edit form with schedule change and DELAYED status.

Figure 13. Oracle output reflecting updated flight 309.

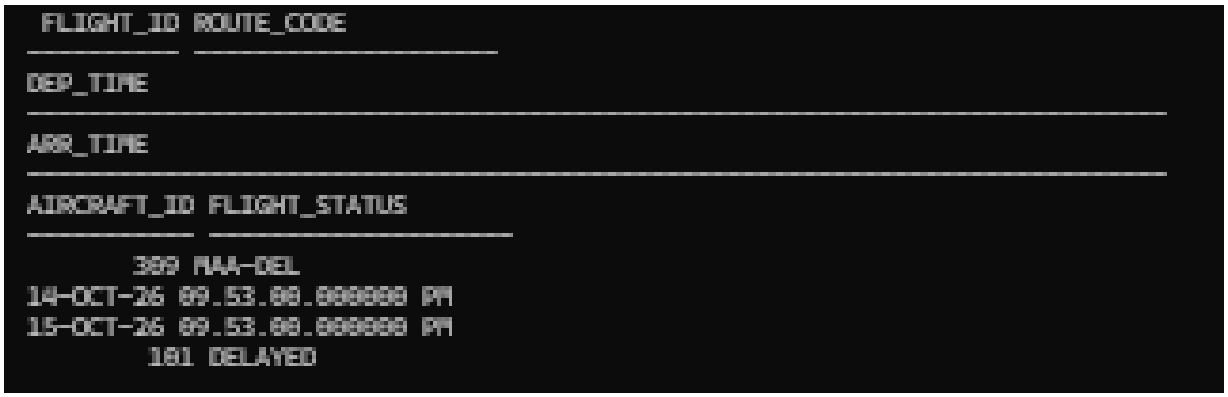


Figure 13. Oracle output reflecting updated flight 309.

Figure 14. Delete confirmation prompt for NS 301; prompt only.

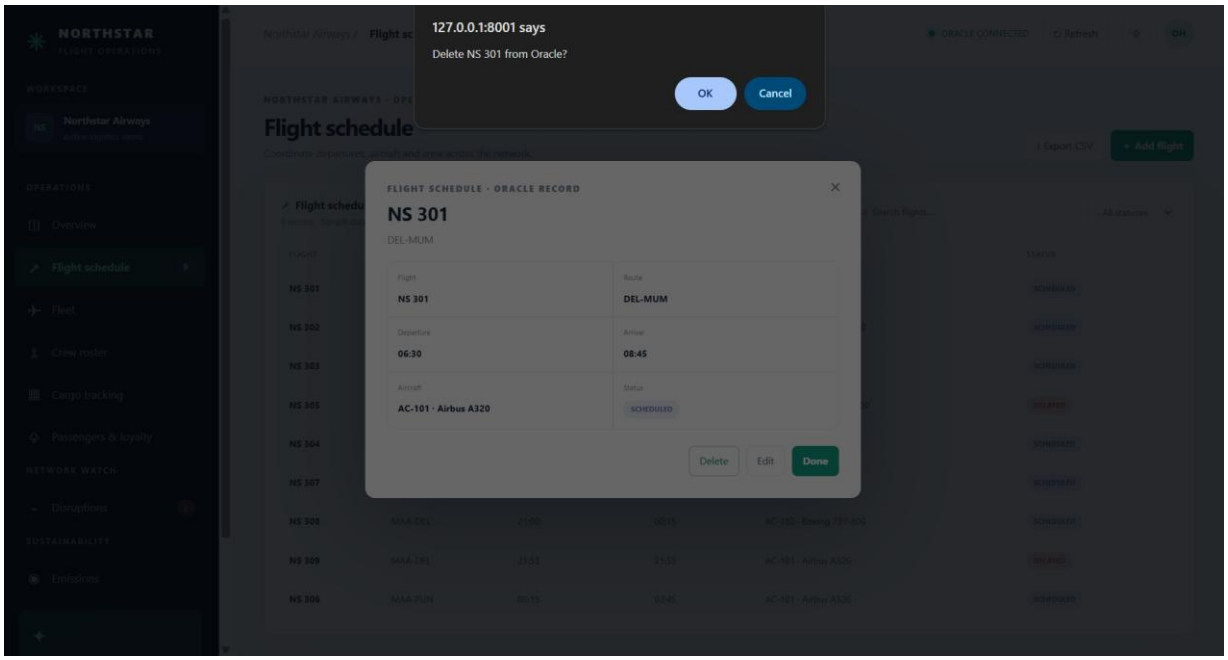


Figure 14. Delete confirmation prompt for NS 301; prompt only.

Figure 15. Fleet module with aircraft and service details.

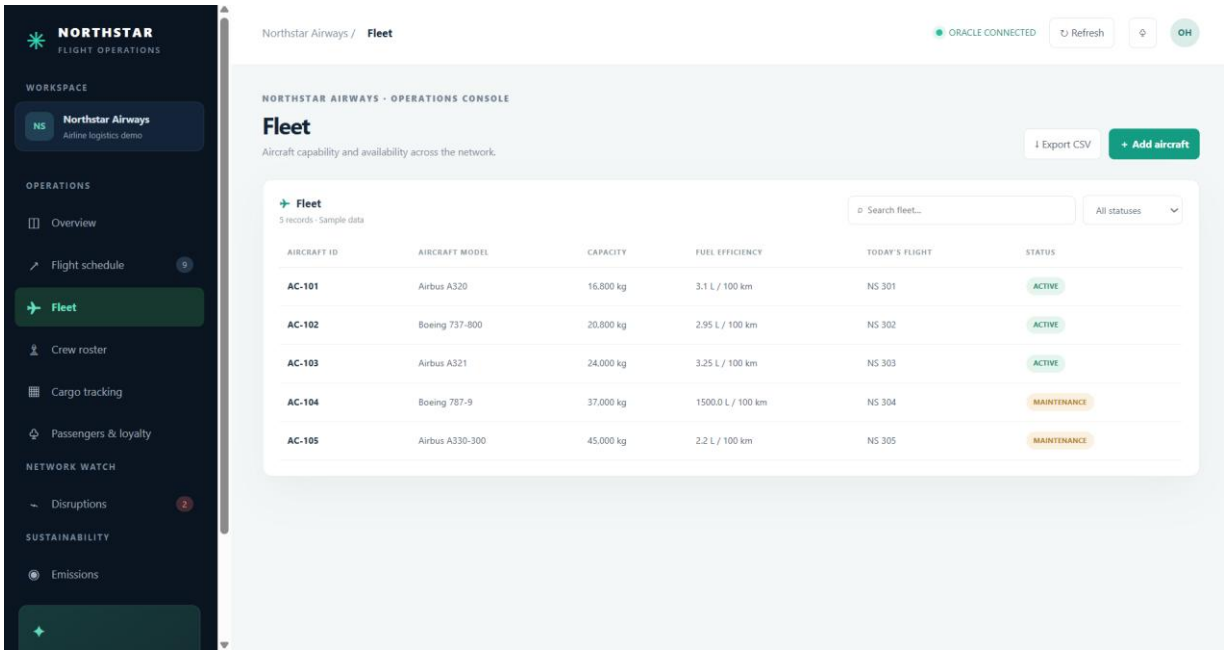


Figure 15. Fleet module with aircraft and service details.

Figure 16. Crew roster with roles, bases and assignments.

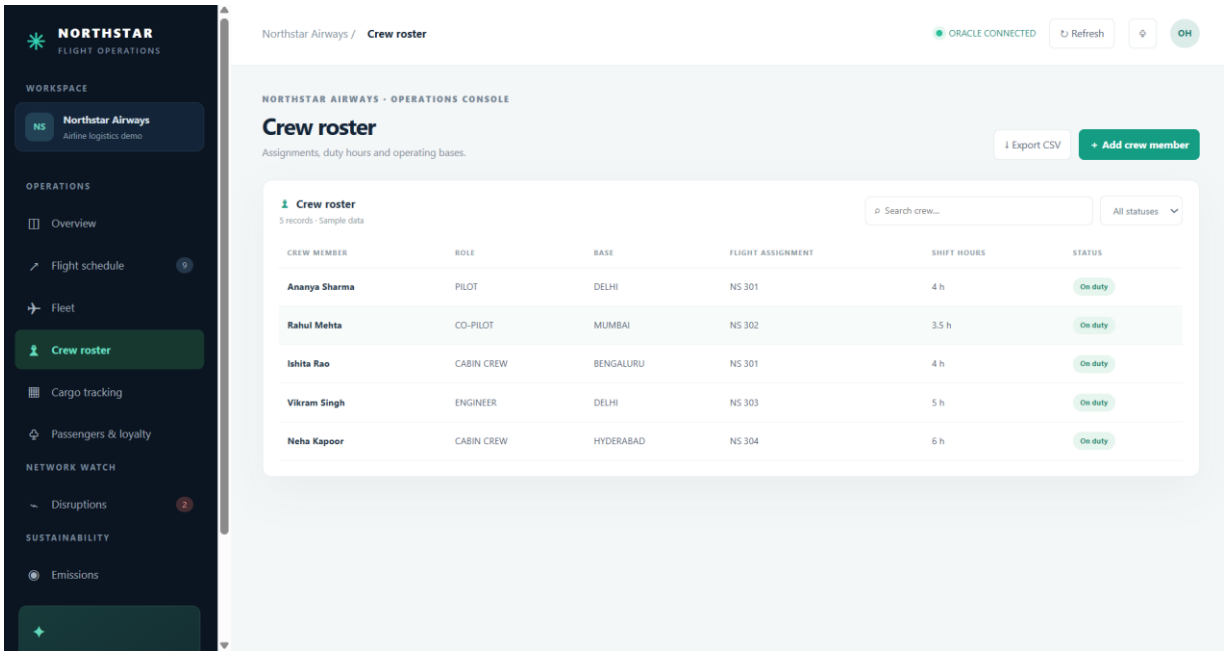


Figure 16. Crew roster with roles, bases and assignments.

Figure 17. Cargo tracking with category and temperature fields.

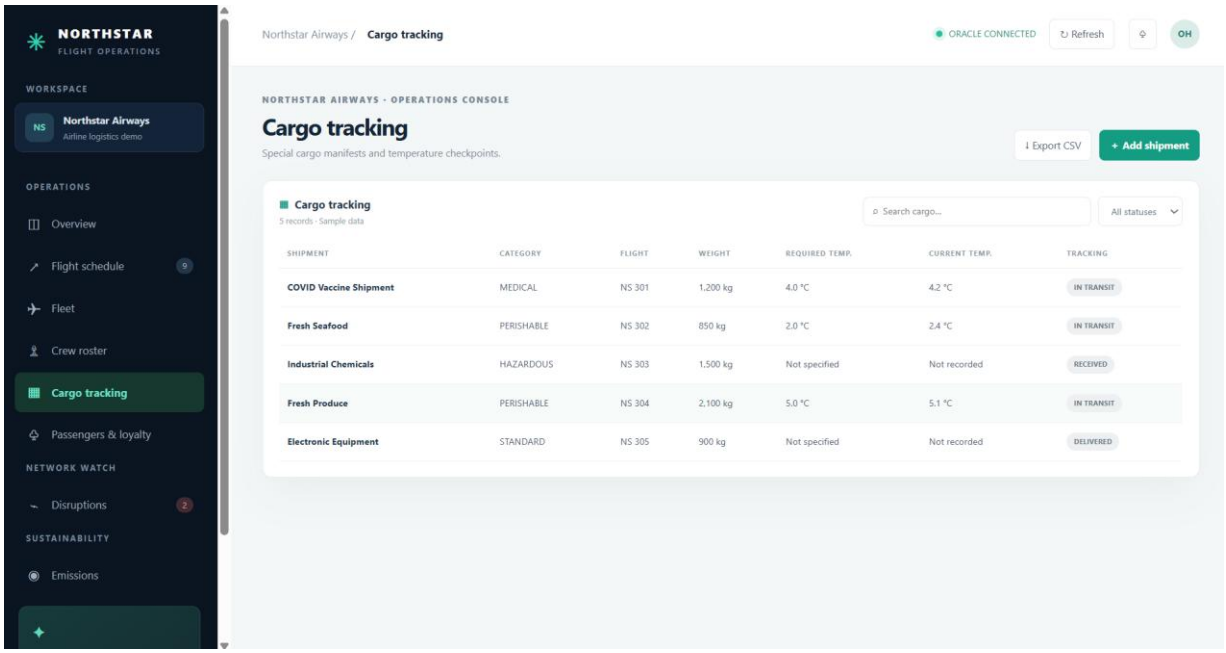


Figure 17. Cargo tracking with category and temperature fields.

Figure 18. Passenger and loyalty module.

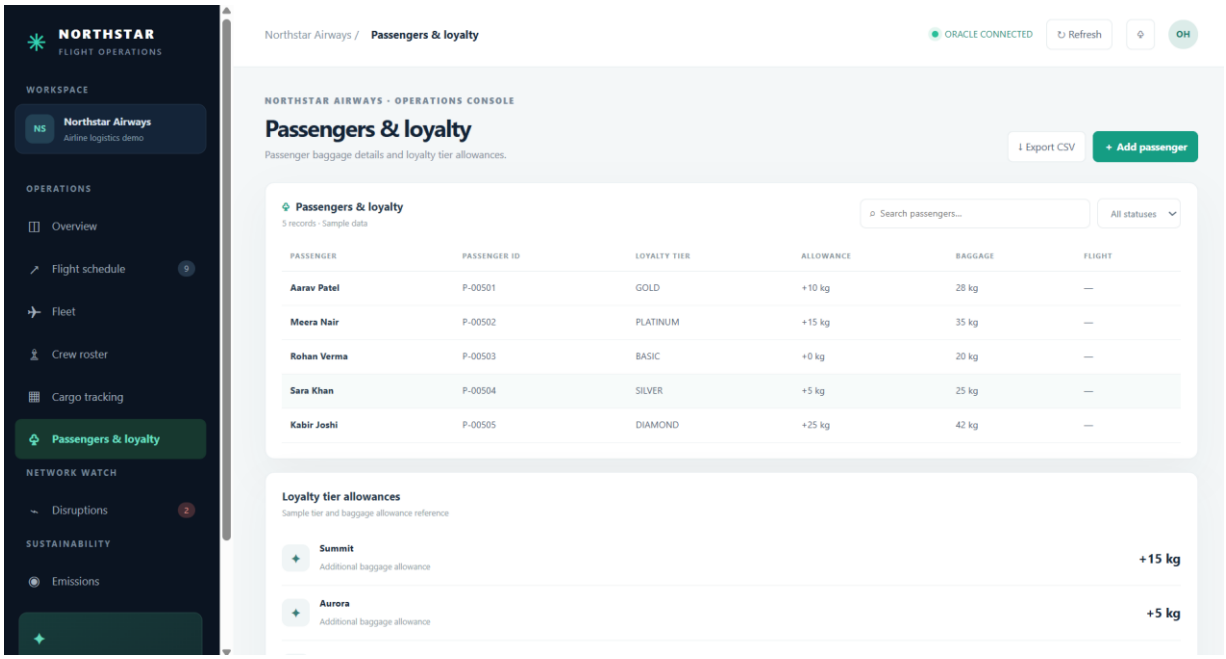


Figure 18. Passenger and loyalty module.

Figure 19. Disruptions module with causes, delay and status.

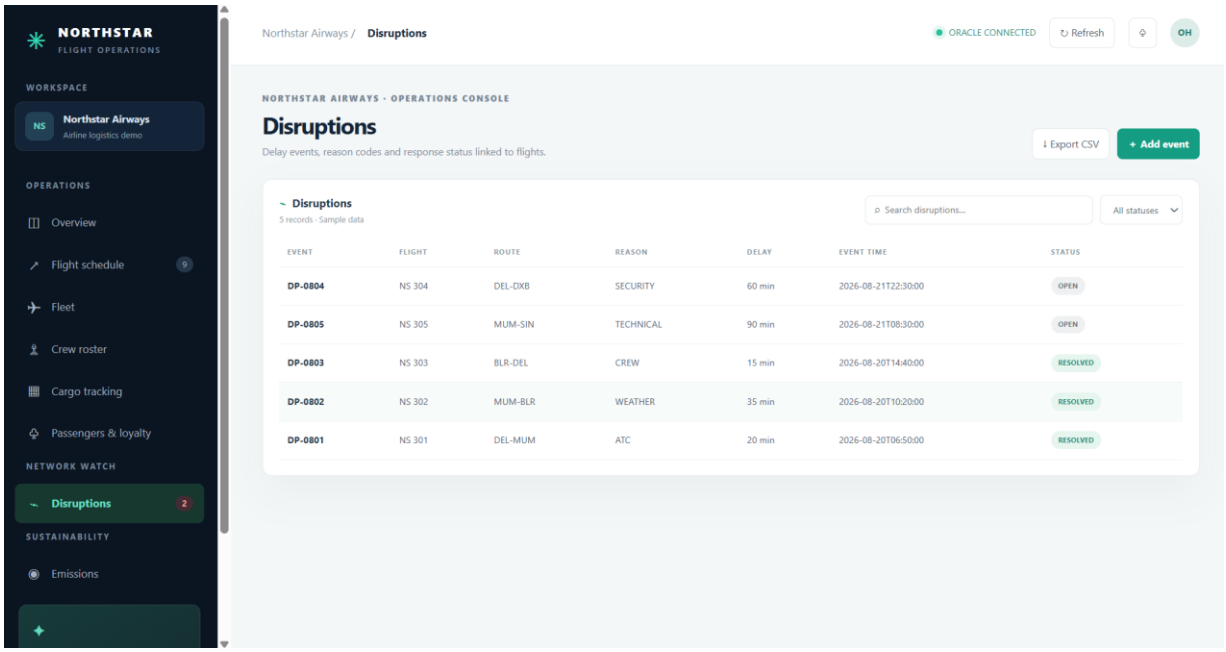


Figure 19. Disruptions module with causes, delay and status.

Figure 20. Emissions module with fuel and CO2 records.

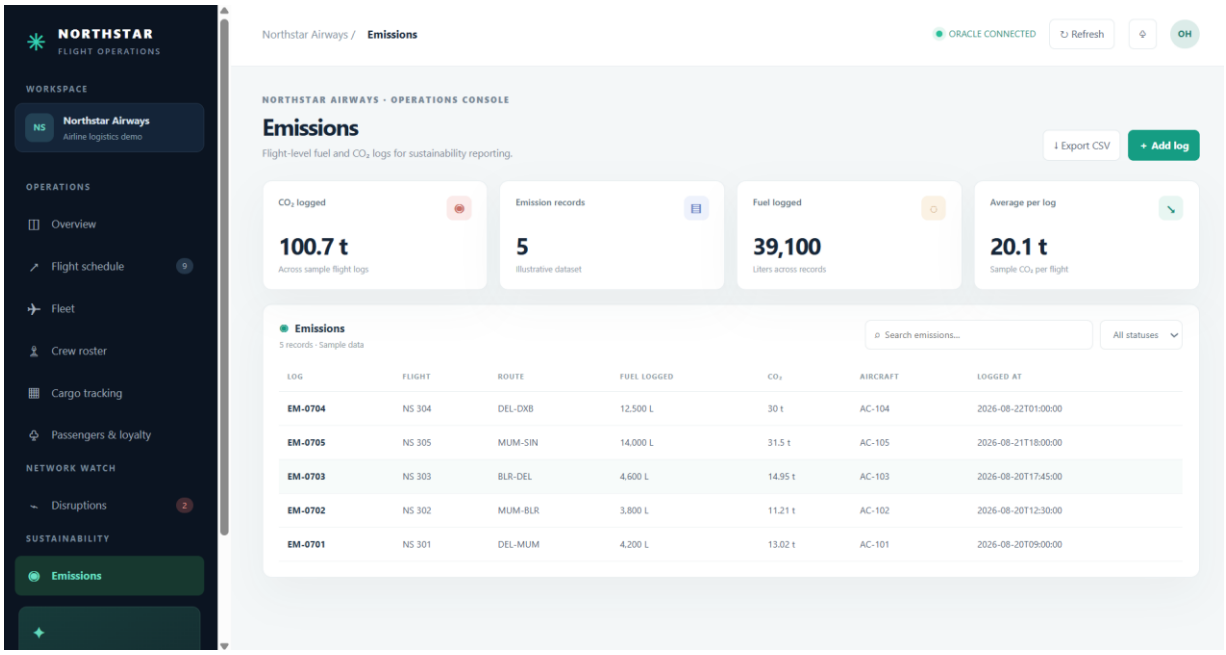


Figure 20. Emissions module with fuel and CO2 records.

Figure 21. Overview after additional flight is present.

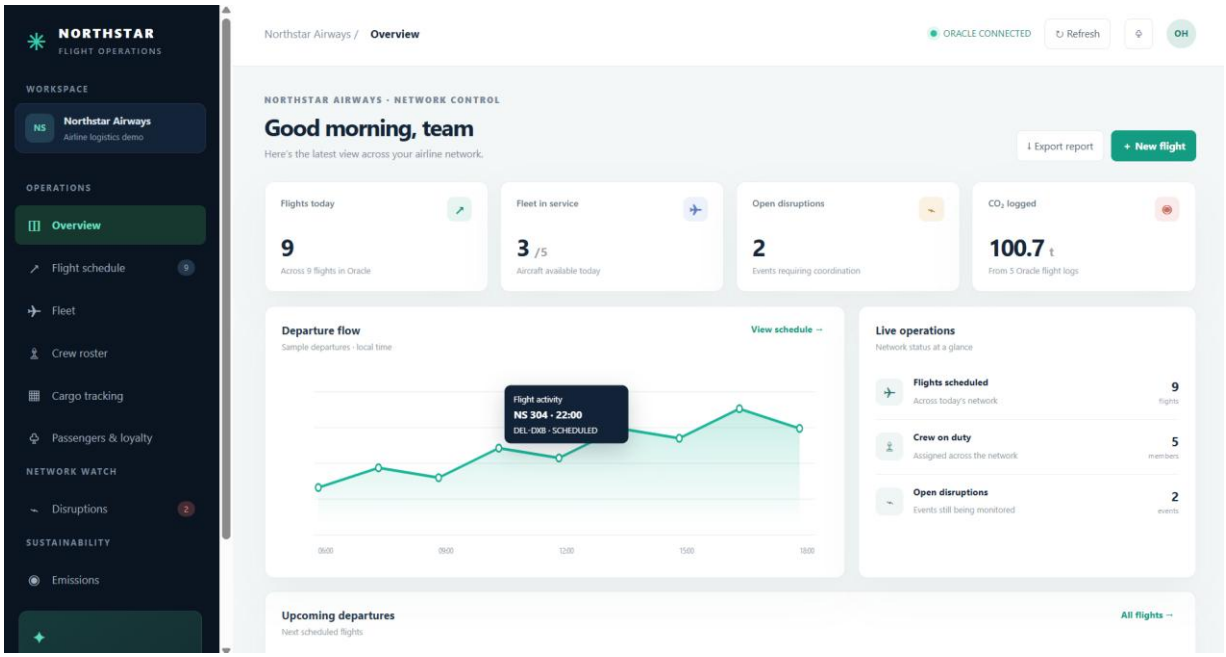


Figure 21. Overview after additional flight is present.

14 CONCLUSION AND FUTURE SCOPE

14.1 Conclusion

The project has progressed from a relational database design in Review 0 to a working local airline-operations application for Review 1. The implementation brings key records into a browser workspace and connects supported CRUD workflows to Oracle XE through FastAPI. Screenshots document authentication, flight schedule operations, database persistence for a create and update example, and fleet, crew, cargo, passenger, disruption and emissions views.

The design provides a foundation for structured operational data and sustainability visibility. It demonstrates database relationships, constraints, SQL concepts and an application layer. It remains a coursework proof of concept: data is illustrative, the service is local-only, permissions are uniform and sustainability outcomes have not been measured.

14.2 Future enhancements

Priority	Enhancement	Expected benefit
1	Role access, session hardening, CSRF protection, TLS and audit logs	Safer deployment; accountability.
2	Validation, allowlists, pagination, transactions and clear errors	Reliable maintenance at higher record volumes.
3	Crew assignment UI and duty/rest rules	Complete staffing workflow; fewer invalid assignments.
4	Sensor ingestion and cold-chain threshold alerts	Timely response and shipment traceability.
5	Verified emissions factors and normalized reporting	Comparable, auditable sustainability analytics.
6	Disruption analytics, alerts and recovery suggestions	Faster coordination and learning from recurring causes.
7	Automated checks, CI, backups, migrations and monitoring	Safer releases and recoverability.

8	Booking, airport and maintenance-system integration	Less duplicate entry; fuller network status.

14.3 Measurable next-step plan

First validate deployed columns and constraints against Oracle metadata. Add API and browser checks for create, update and delete, including unauthorized access and database constraint failures. Add roles and audit records before network deployment. Pilot emissions and cargo measurements with timestamped, attributed sources, then evaluate completeness, exception response time and normalized fuel/emissions indicators.

REFERENCES AND PROJECT MATERIALS

Review 0 report: Modern Airline Logistics and Sustainability Management System, BACSE202 Lab Component Project, AY 2026–27.

Review 1 guideline: Database Systems QP 9 (2).pdf, School of Computer Science and Engineering (SCOPE), AY 2026–27.

Project source files: server.py, index.html, requirements.txt and README.txt.

Implementation evidence: application and Oracle screenshots supplied by the author on 13 October 2026.

APPENDIX A FULL IMPLEMENTATION CODE SERVER

Listing reproduced from server.py. Private .env and session-secret files are intentionally excluded.

```
0001  import os
0002  import base64
0003  import hashlib
0004  import hmac
0005  import json
0006  import re
0007  import secrets
0008  from contextlib import contextmanager
0009  from datetime import date, datetime
0010  from pathlib import Path
0011  from typing import Any
0012
0013  import oracledb
0014  from dotenv import load_dotenv
0015  from fastapi import Depends, FastAPI, HTTPException, Request, Response
0016  from fastapi.responses import FileResponse
0017
0018  ROOT = Path(__file__).resolve().parent
0019  load_dotenv(ROOT / ".env")
0020
0021  app = FastAPI(title="Northstar Oracle API")
0022  _pool: oracledb.ConnectionPool | None = None
0023  AUTH_TABLE = "APP_LOGIN_USERS"
0024  SESSION_COOKIE = "northstar_session"
0025  SESSION_SECONDS = 12 * 60 * 60
0026  _session_secret: bytes | None = None
0027
0028  ENTITIES: dict[str, dict[str, Any]] = {
0029      "flights": {
0030          "table": "FLIGHT", "key": "FLIGHT_ID",
0031          "fields": {"route": "ROUTE_CODE", "departure": "DEP_TIME", "arrival": "ARR_TIME", "aircraft_id": "AIRCRAFT_ID", "status":
"FLIGHT_STATUS"},
0032      },
0033      "fleet": {
0034          "table": "AIRCRAFT", "key": "AIRCRAFT_ID",
0035          "fields": {"model": "MODEL", "capacity_kg": "CAPACITY_KG", "efficiency": "FUEL_EFFICIENCY", "status": "AIRCRAFT_STATUS"},
0036      },
0037      "crew": {
0038          "table": "CREW", "key": "CREW_ID",
0039          "fields": {"name": "CREW_NAME", "role": "ROLE", "base": "BASE_LOCATION"},
0040      },
0041      "cargo": {
0042          "table": "SPEC_CARGO", "key": "CARGO_ID",
0043          "fields": {"flight_id": "FLIGHT_ID", "name": "CARGO_DESCRIPTION", "weight_kg": "WEIGHT_KG", "category": "CATEGORY",
"required_temp_c": "REQ_TEMP_C", "current_temp_c": "CURRENT_TEMP_C", "tracking": "TRACKING_STATUS"},
0044      },
0045      "passengers": {
0046          "table": "PASSENGER", "key": "PASSENGER_ID",
0047          "fields": {"name": "PASSENGER_NAME", "passport_no": "PASSPORT_NO", "tier_id": "TIER_ID", "baggage_kg": "BAGGAGE_KG"},
0048      },
0049      "disruptions": {
0050          "table": "DISRUPTION", "key": "EVENT_ID",
0051          "fields": {"flight_id": "FLIGHT_ID", "delay_min": "DELAY_MIN", "reason": "REASON_CODE", "event_date": "EVENT_DATE",
"status": "EVENT_STATUS"},
0052      },
0053      "emissions": {
0054          "table": "EMISSIONS", "key": "LOG_ID",
0055          "fields": {"flight_id": "FLIGHT_ID", "fuel_liters": "FUEL_LITERS", "co2_tons": "CO2_TONS", "logged_at": "LOGGED_AT"},
0056      },
0057  }
0058
0059
0060  def get_pool() -> oracledb.ConnectionPool:
0061      global _pool
0062      if _pool is None:
0063          user = os.getenv("ORACLE_USER")
0064          password = os.getenv("ORACLE_PASSWORD")
0065          dsn = os.getenv("ORACLE_DSN")
0066          if not user or not password or not dsn:
0067              raise HTTPException(status_code=503, detail="Set ORACLE_USER, ORACLE_PASSWORD, and ORACLE_DSN in the local .env file.")
0068          try:
0069              _pool = oracledb.create_pool(user=user, password=password, dsn=dsn, min=1, max=4, increment=1)
0070          except oracledb.Error as exc:
0071              raise HTTPException(status_code=503, detail=f"Oracle connection failed: {exc}") from exc
0072      return _pool
0073
0074
0075  @contextmanager
0076  def connection():
0077      try:
0078          conn = get_pool().acquire()
0079      except oracledb.Error as exc:
0080          raise HTTPException(status_code=503, detail=f"Cannot acquire an Oracle connection: {exc}") from exc
0081      try:
```

```
        yield conn
    finally:
        conn.close()


def rows_as_dicts(cursor: oracledb.Cursor) -> list[dict[str, Any]]:
    names = [column[0].lower() for column in cursor.description]
    result = []
    for row in cursor:
        item = dict(zip(names, row))
        for key, value in item.items():
            if isinstance(value, (datetime, date)):
                item[key] = value.isoformat()
        result.append(item)
    return result


def ensure_auth_table() -> None:
    with connection() as conn:
        with conn.cursor() as cursor:
            cursor.execute("SELECT COUNT(*) FROM USER_TABLES WHERE TABLE_NAME = :table_name", {"table_name": AUTH_TABLE})
            if cursor.fetchone()[0]:
                return
            try:
                cursor.execute(f"""CREATE TABLE {AUTH_TABLE} (
                    USER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
                    NAME VARCHAR2(80) NOT NULL,
                    EMAIL VARCHAR2(254) NOT NULL UNIQUE,
                    PASSWORD_HASH VARCHAR2(256) NOT NULL,
                    CREATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
                )""")
            except oracledb.DatabaseError as exc:
                if getattr(exc.args[0], "code", None) != 955:
                    raise


def hash_password(password: str, salt: bytes | None = None) -> str:
    salt = salt or secrets.token_bytes(16)
    digest = hashlib.scrypt(password.encode("utf-8"), salt=salt, n=2**14, r=8, p=1, dklen=64)
    return f"scrypt$16384$8$1${base64.urlsafe_b64encode(salt).decode()}${base64.urlsafe_b64encode(digest).decode()}"


def verify_password(password: str, encoded: str) -> bool:
    try:
        algorithm, n, r, p, salt_text, digest_text = encoded.split("$", 5)
        if algorithm != "scrypt":
            return False
        salt = base64.urlsafe_b64decode(salt_text.encode())
        expected = base64.urlsafe_b64decode(digest_text.encode())
        actual = hashlib.scrypt(password.encode("utf-8"), salt=salt, n=int(n), r=int(r), p=int(p), dklen=len(expected))
        return hmac.compare_digest(actual, expected)
    except (ValueError, TypeError):
        return False


def get_session_secret() -> bytes:
    global _session_secret
    if _session_secret is None:
        path = ROOT / ".session_secret"
        try:
            _session_secret = path.read_bytes()
        except FileNotFoundError:
            _session_secret = secrets.token_bytes(32)
            try:
                with path.open("xb") as secret_file:
                    secret_file.write(_session_secret)
            except FileExistsError:
                _session_secret = path.read_bytes()
    return _session_secret


def create_session(user: dict[str, Any]) -> str:
    payload = {"id": user["user_id"], "name": user["name"], "email": user["email"], "exp": int(datetime.now().timestamp()) +
SESSION_SECONDS}
    encoded = base64.urlsafe_b64encode(json.dumps(payload, separators=(",", ":")).encode()).rstrip(b"=")
    signature = hmac.new(get_session_secret(), encoded, hashlib.sha256).digest()
    return f"{encoded.decode()}.{base64.urlsafe_b64encode(signature).rstrip(b'=').decode()}"


def session_user(request: Request) -> dict[str, Any] | None:
    token = request.cookies.get(SESSION_COOKIE)
    if not token or "." not in token:
        return None
    encoded_text, signature_text = token.rsplit(".", 1)
    encoded = encoded_text.encode()
    expected = hmac.new(get_session_secret(), encoded, hashlib.sha256).digest()
    try:
        supplied = base64.urlsafe_b64decode(signature_text + "=" * (-len(signature_text) % 4))
        payload_bytes = base64.urlsafe_b64decode(encoded_text + "=" * (-len(encoded_text) % 4))
        payload = json.loads(payload_bytes)
```

```
0171      except (ValueError, json.JSONDecodeError):
0172          return None
0173      if not hmac.compare_digest(expected, supplied) or payload.get("exp", 0) < datetime.now().timestamp():
0174          return None
0175      return payload
0176
0177
0178  def require_user(request: Request) -> dict[str, Any]:
0179      user = session_user(request)
0180      if user is None:
0181          raise HTTPException(status_code=401, detail="Please sign in to continue.")
0182      return user
0183
0184
0185  def flight_label(flight_id: int | None) -> str:
0186      return f"NS {flight_id:03d}" if flight_id is not None else "—"
0187
0188
0189  def read_all() -> dict[str, list[dict[str, Any]]]:
0190      queries = {
0191          "flights": """SELECT f.FLIGHT_ID AS db_id, f.ROUTE_CODE AS route,
0192              TO_CHAR(f.DEP_TIME, 'YYYY-MM-DD\"T\"HH24:MI:SS') AS departure_iso,
0193              TO_CHAR(f.DEP_TIME, 'HH24:MI') AS departure,
0194              TO_CHAR(f.ARR_TIME, 'YYYY-MM-DD\"T\"HH24:MI:SS') AS arrival_iso,
0195              TO_CHAR(f.ARR_TIME, 'HH24:MI') AS arrival,
0196              f.AIRCRAFT_ID AS aircraft_id, a.MODEL AS model, f.FLIGHT_STATUS AS status
0197              FROM FLIGHT f JOIN AIRCRAFT a ON a.AIRCRAFT_ID = f.AIRCRAFT_ID ORDER BY f.DEP_TIME""",
0198          "fleet": """SELECT a.AIRCRAFT_ID AS db_id, a.MODEL AS model, a.CAPACITY_KG AS capacity_kg,
0199              a.FUEL_EFFICIENCY AS efficiency, a.AIRCRAFT_STATUS AS status,
0200              (SELECT MIN(f.FLIGHT_ID) FROM FLIGHT f WHERE f.AIRCRAFT_ID = a.AIRCRAFT_ID) AS flight_id
0201              FROM AIRCRAFT a ORDER BY a.AIRCRAFT_ID""",
0202          "crew": """SELECT c.CREW_ID AS db_id, c.CREW_NAME AS name, c.ROLE AS role,
0203              c.BASE_LOCATION AS base, fc.FLIGHT_ID AS flight_id, fc.SHIFT_HOURS AS shift
0204              FROM CREW c LEFT JOIN (SELECT CREW_ID, MIN(FLIGHT_ID) AS FLIGHT_ID,
0205              MIN(SHIFT_HOURS) AS SHIFT_HOURS FROM FLIGHT_CREW GROUP BY CREW_ID) fc
0206              ON fc.CREW_ID = c.CREW_ID ORDER BY c.CREW_ID""",
0207          "cargo": """SELECT c.CARGO_ID AS db_id, c.CARGO_DESCRIPTION AS name, c.CATEGORY AS category,
0208              c.FLIGHT_ID AS flight_id, c.WEIGHT_KG AS weight_kg, c.REQ_TEMP_C AS required_temp_c,
0209              c.CURRENT_TEMP_C AS current_temp_c, c.TRACKING_STATUS AS tracking
0210              FROM SPEC_CARGO c ORDER BY c.CARGO_ID""",
0211          "passengers": """SELECT p.PASSENGER_ID AS db_id, p.PASSENGER_NAME AS name,
0212              p.TIER_ID AS tier_id, lt.TIER_NAME AS tier, lt.EXTRA_BAGGAGE_KG AS allowance_kg,
0213              p.BAGGAGE_KG AS baggage_kg,
0214              (SELECT MIN(b.FLIGHTNO) FROM BOOKING b WHERE b.P_ID = TO_CHAR(p.PASSENGER_ID)) AS flight_id
0215              FROM PASSENGER p JOIN LOYALTY_TIER lt ON lt.TIER_ID = p.TIER_ID ORDER BY p.PASSENGER_ID""",
0216          "disruptions": """SELECT d.EVENT_ID AS db_id, d.FLIGHT_ID AS flight_id, f.ROUTE_CODE AS route,
0217              d.REASON_CODE AS reason, d.DELAY_MIN AS delay_min,
0218              TO_CHAR(d.EVENT_DATE, 'YYYY-MM-DD\"T\"HH24:MI:SS') AS event_date,
0219              d.EVENT_STATUS AS status FROM DISRUPTION d JOIN FLIGHT f ON f.FLIGHT_ID = d.FLIGHT_ID
0220              ORDER BY d.EVENT_DATE DESC""",
0221          "emissions": """SELECT e.LOG_ID AS db_id, e.FLIGHT_ID AS flight_id, f.ROUTE_CODE AS route,
0222              e.FUEL_LITERS AS fuel_liters, e.CO2_TONS AS co2_tons, f.AIRCRAFT_ID AS aircraft_id,
0223              TO_CHAR(e.LOGGED_AT, 'YYYY-MM-DD\"T\"HH24:MI:SS') AS logged_at
0224              FROM EMISSIONS e JOIN FLIGHT f ON f.FLIGHT_ID = e.FLIGHT_ID ORDER BY e.LOGGED_AT DESC""",
0225          "loyalty_tiers": "SELECT TIER_ID AS tier_id, TIER_NAME AS tier_name, EXTRA_BAGGAGE_KG AS allowance_kg FROM LOYALTY_TIER
ORDER BY TIER_ID",
0226      }
0227      result: dict[str, list[dict[str, Any]]] = {}
0228      with connection() as conn:
0229          with conn.cursor() as cursor:
0230              for entity, sql in queries.items():
0231                  cursor.execute(sql)
0232                  records = rows_as_dicts(cursor)
0233                  if entity == "flights":
0234                      for row in records:
0235                          row.update(id=flight_label(row["db_id"]), aircraft=f"AC-{row['aircraft_id']:03d} · {row['model']}")
0236                  elif entity == "fleet":
0237                      for row in records:
0238                          row.update(id=f"AC-{row['db_id']:03d}", capacity=f"{row['capacity_kg']:,.0f} kg",
efficiency=f"{row['efficiency']} L / 100 km", flights=flight_label(row["flight_id"]) if row["flight_id"] else "—")
0239                  elif entity == "crew":
0240                      for row in records:
0241                          row.update(id=f"CR-{row['db_id']:03d}", flight=flight_label(row["flight_id"]) if row["flight_id"] else "—",
shift=f"{row['shift']} h" if row["shift"] is not None else "—", status="On duty" if row["flight_id"] else "Standby")
0242                  elif entity == "cargo":
0243                      for row in records:
0244                          row.update(id=f"CG-{row['db_id']:04d}", flight=flight_label(row["flight_id"]),
weight=f"{row['weight_kg']:,.0f} kg", required=f"{row['required_temp_c']} °C" if row["required_temp_c"] is not None else "Not specified",
current=f"{row['current_temp_c']} °C" if row["current_temp_c"] is not None else "Not recorded")
0245                  elif entity == "passengers":
0246                      for row in records:
0247                          row.update(id=f"P-{row['db_id']:05d}", allowance=f"+{row['allowance_kg']:g} kg",
baggage=f"{row['baggage_kg']:g} kg", flight=flight_label(row["flight_id"]) if row["flight_id"] else "—")
0248                  elif entity == "disruptions":
0249                      for row in records:
0250                          row.update(id=f"DP-{row['db_id']:04d}", flight=flight_label(row["flight_id"]), delay=f"{row['delay_min']}
min", date=row["event_date"])
0251                  elif entity == "emissions":
0252                      for row in records:
```

```
                        row.update(id=f"EM-{row['db_id']:04d}", flight=flight_label(row["flight_id"]),
fuel=f"{row['fuel_liters']:,.0f} L", co2=f"{row['co2_tons']:g} t", aircraft=f"AC-{row['aircraft_id']:03d}", logged=row["logged_at"])
                result[entity] = records
    return result


@app.get("/api/health")
def health() -> dict[str, Any]:
    with connection() as conn:
        with conn.cursor() as cursor:
            cursor.execute("SELECT USER, SYS_CONTEXT('USERENV','SERVICE_NAME'), SYS_CONTEXT('USERENV','CON_NAME') FROM DUAL")
            user, service, container = cursor.fetchone()
            cursor.execute("SELECT TABLE_NAME FROM USER_TABLES")
            visible = {row[0] for row in cursor.fetchall()}
    required = {"AIRCRAFT", "CREW", "DISRUPTION", "EMISSIONS", "FLIGHT", "LOYALTY_TIER", "PASSENGER", "SPEC_CARGO"}
    missing = sorted(required - visible)
    if missing:
        raise HTTPException(status_code=503, detail=f"Oracle is reachable as {user} on {service}/{container}, but this schema cannot
see the airline tables. Use the same Oracle username and service as the SQL*Plus session where you ran the inserts.")
    return {"status": "ok", "database": "Oracle", "user": user, "service": service, "container": container}


@app.post("/api/auth/signup", status_code=201)
def signup(payload: dict[str, Any], response: Response) -> dict[str, Any]:
    name = str(payload.get("name", "")).strip()
    email = str(payload.get("email", "")).strip().lower()
    password = str(payload.get("password", ""))
    if not name or len(name) > 80:
        raise HTTPException(status_code=422, detail="Name is required and must be 80 characters or fewer.")
    if not re.fullmatch(r"[^\s@]+@[^\s@]+\.[^\s@]+", email) or len(email) > 254:
        raise HTTPException(status_code=422, detail="Enter a valid email address.")
    if len(password) < 10:
        raise HTTPException(status_code=422, detail="Password must be at least 10 characters.")
    ensure_auth_table()
    try:
        with connection() as conn:
            with conn.cursor() as cursor:
                user_id_var = cursor.var(oracledb.NUMBER)
                cursor.execute(
                    f"INSERT INTO {AUTH_TABLE} (NAME, EMAIL, PASSWORD_HASH) VALUES (:name, :email, :password_hash) RETURNING USER_ID
INTO :user_id",
                    {"name": name, "email": email, "password_hash": hash_password(password), "user_id": user_id_var},
                )
                user_id = user_id_var.getvalue()
                conn.commit()
    except oracledb.DatabaseError as exc:
        if getattr(exc.args[0], "code", None) == 1:
            raise HTTPException(status_code=409, detail="An account with this email already exists.") from exc
        raise HTTPException(status_code=503, detail="Could not create the account in Oracle.") from exc
    if isinstance(user_id, list):
        user_id = user_id[0]
    user = {"user_id": int(user_id), "name": name, "email": email}
    response.set_cookie(SESSION_COOKIE, create_session(user), httponly=True, samesite="lax", secure=False, max_age=SESSION_SECONDS,
path="/")
    return user


@app.post("/api/auth/login")
def login(payload: dict[str, Any], response: Response) -> dict[str, Any]:
    email = str(payload.get("email", "")).strip().lower()
    password = str(payload.get("password", ""))
    ensure_auth_table()
    try:
        with connection() as conn:
            with conn.cursor() as cursor:
                cursor.execute(f"SELECT USER_ID, NAME, EMAIL, PASSWORD_HASH FROM {AUTH_TABLE} WHERE EMAIL = :email", {"email":
email})
                row = cursor.fetchone()
    except oracledb.DatabaseError as exc:
        raise HTTPException(status_code=503, detail="Could not check the account in Oracle.") from exc
    if row is None or not verify_password(password, row[3]):
        raise HTTPException(status_code=401, detail="Email or password is incorrect.")
    user = {"user_id": int(row[0]), "name": row[1], "email": row[2]}
    response.set_cookie(SESSION_COOKIE, create_session(user), httponly=True, samesite="lax", secure=False, max_age=SESSION_SECONDS,
path="/")
    return user


@app.get("/api/auth/me")
def current_user(user: dict[str, Any] | None = Depends(session_user)) -> dict[str, Any]:
    if user is None:
        raise HTTPException(status_code=401, detail="Please sign in to continue.")
    return user


@app.post("/api/auth/logout")
def logout(response: Response) -> dict[str, str]:
    response.delete_cookie(SESSION_COOKIE, path="/")
    return {"message": "Signed out"}

```

```
0337
0338  @app.get("/api/data")
0339  def get_data(_user: dict[str, Any] = Depends(require_user)) -> dict[str, list[dict[str, Any]]]:
0340      try:
0341          return read_all()
0342      except oracledb.DatabaseError as exc:
0343          error = exc.args[0]
0344          if getattr(error, "code", None) == 942:
0345              raise HTTPException(status_code=503, detail="Oracle is reachable, but the configured schema cannot see one or more
airline tables. Use the same Oracle username and service as the SQL*Plus session where you ran the inserts.") from exc
0346          raise HTTPException(status_code=503, detail=f"Oracle query failed: {error}") from exc
0347
0348
0349  @app.get("/api/reference/{entity}")
0350  def get_reference(entity: str, _user: dict[str, Any] = Depends(require_user)) -> list[dict[str, Any]]:
0351      queries = {
0352          "flights": "SELECT FLIGHT_ID AS db_id, ROUTE_CODE AS route FROM FLIGHT ORDER BY FLIGHT_ID",
0353          "fleet": "SELECT AIRCRAFT_ID AS db_id, MODEL AS model FROM AIRCRAFT ORDER BY AIRCRAFT_ID",
0354          "tiers": "SELECT TIER_ID AS tier_id, TIER_NAME AS tier_name FROM LOYALTY_TIER ORDER BY TIER_ID",
0355      }
0356      sql = queries.get(entity)
0357      if sql is None:
0358          raise HTTPException(status_code=404, detail="Unknown reference list.")
0359      with connection() as conn:
0360          with conn.cursor() as cursor:
0361              cursor.execute(sql)
0362              return rows_as_dicts(cursor)
0363
0364
0365  def get_entity(entity: str) -> dict[str, Any]:
0366      definition = ENTITIES.get(entity)
0367      if definition is None:
0368          raise HTTPException(status_code=404, detail="Unknown entity.")
0369      return definition
0370
0371
0372  def write_record(entity: str, key_value: int | str | None, payload: dict[str, Any], create: bool) -> dict[str, Any]:
0373      definition = get_entity(entity)
0374      fields: dict[str, str] = definition["fields"]
0375      provided_key = payload.pop("db_id", None) if create else None
0376      unknown = set(payload) - set(fields)
0377      if unknown:
0378          raise HTTPException(status_code=422, detail=f"Unsupported fields: {', '.join(sorted(unknown))}")
0379      if provided_key is not None:
0380          try:
0381              provided_key = int(provided_key)
0382          except (TypeError, ValueError) as exc:
0383              raise HTTPException(status_code=422, detail="db_id must be a number.") from exc
0384      if not payload:
0385          raise HTTPException(status_code=422, detail="Provide at least one field.")
0386      columns = list(payload)
0387      binds = {name: payload[name] for name in columns}
0388      date_fields = {"departure", "arrival", "event_date", "logged_at"}
0389      for name in columns:
0390          if name in date_fields and binds[name] is not None:
0391              try:
0392                  binds[name] = datetime.fromisoformat(str(binds[name]))
0393              except ValueError as exc:
0394                  raise HTTPException(status_code=422, detail=f"{name} must be an ISO date/time.") from exc
0395      try:
0396          with connection() as conn:
0397              with conn.cursor() as cursor:
0398                  if create:
0399                      column_sql = ", ".join(fields[name] for name in columns)
0400                      bind_sql = ", ".join(f":{name}" for name in columns)
0401                      if provided_key is not None:
0402                          column_sql = f"{definition['key']}, {column_sql}"
0403                          bind_sql = f":provided_key, {bind_sql}"
0404                          binds["provided_key"] = provided_key
0405                      key_var = cursor.var(oracledb.NUMBER)
0406                      sql = f"INSERT INTO {definition['table']} ({column_sql}) VALUES ({bind_sql}) RETURNING {definition['key']} INTO
:new_key"
0407                      cursor.execute(sql, {**binds, "new_key": key_var})
0408                      new_key = key_var.getvalue()
0409                      conn.commit()
0410                      return {"db_id": new_key, "message": "Created"}
0411                  assignments = ", ".join(f"{fields[name]} = :{name}" for name in columns)
0412                  binds["record_key"] = key_value
0413                  cursor.execute(f"UPDATE {definition['table']} SET {assignments} WHERE {definition['key']} = :record_key", binds)
0414                  if cursor.rowcount == 0:
0415                      raise HTTPException(status_code=404, detail="Record not found.")
0416                  conn.commit()
0417                  return {"db_id": key_value, "message": "Updated"}
0418      except HTTPException:
0419          raise
0420      except oracledb.Error as exc:
0421          raise HTTPException(status_code=409, detail=f"Oracle rejected the change. Check required fields, relationships, and
generated IDs: {exc}") from exc
0422
0423
```

```
@app.post("/api/{entity}", status_code=201)
def create_record(entity: str, payload: dict[str, Any], _user: dict[str, Any] = Depends(require_user)) -> dict[str, Any]:
    return write_record(entity, None, payload, True)


@app.put("/api/{entity}/{record_id}")
def update_record(entity: str, record_id: str, payload: dict[str, Any], _user: dict[str, Any] = Depends(require_user)) -> dict[str,
Any]:
    definition = get_entity(entity)
    try:
        key_value: int | str = int(record_id) if definition["key"] != "BOOKING_ID" else record_id
    except ValueError as exc:
        raise HTTPException(status_code=422, detail="Record ID must be numeric.") from exc
    return write_record(entity, key_value, payload, False)


@app.delete("/api/{entity}/{record_id}")
def delete_record(entity: str, record_id: str, _user: dict[str, Any] = Depends(require_user)) -> dict[str, str]:
    definition = get_entity(entity)
    try:
        key_value = int(record_id)
    except ValueError as exc:
        raise HTTPException(status_code=422, detail="Record ID must be numeric.") from exc
    try:
        with connection() as conn:
            with conn.cursor() as cursor:
                cursor.execute(f"DELETE FROM {definition['table']} WHERE {definition['key']} = :record_key", {"record_key":
key_value})
                if cursor.rowcount == 0:
                    raise HTTPException(status_code=404, detail="Record not found.")
                conn.commit()
        return {"message": "Deleted"}
    except HTTPException:
        raise
    except oracledb.Error as exc:
        raise HTTPException(status_code=409, detail=f"Oracle rejected the delete, possibly because related records exist: {exc}")
from exc


@app.get("/")
def index() -> FileResponse:
    return FileResponse(ROOT / "index.html")


@app.get("/{asset_path:path}")
def static_files(asset_path: str) -> FileResponse:
    path = (ROOT / asset_path).resolve()
    if ROOT not in path.parents or not path.is_file():
        raise HTTPException(status_code=404, detail="Not found.")
    return FileResponse(path)
```

APPENDIX B FULL IMPLEMENTATION CODE FRONT END

Listing reproduced from index.html. Private .env and session-secret files are intentionally excluded.

```
0001  <!doctype html>
0002  <html lang="en">
0003  <head>
0004  <meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1"><meta name="theme-color"
content="#0b1421">
0005  <title>Northstar · Airline Logistics</title>
0006  <style>
0007  :root{--nav:#0b1421;--nav2:#121f30;--ink:#172638;--muted:#7f8b98;--line:#e7edf0;--bg:#f4f7f8;--teal:#17b394;--teal2:#e7f7f3;--
amber:#d99438;--red:#d36c62;--shadow:0 18px 45px #1726380b}*{box-sizing:border-box}body{margin:0;background:var(--bg);font:14px/1.45
"Segoe UI",Arial,sans-serif;color:var(--ink)}button,input,select{font:inherit}button{cursor:pointer}.layout{display:grid;grid-template-
columns:252px 1fr;min-height:100vh}.sidebar{background:var(--nav);color:#dbe4ed;padding:22px 15px 16px;display:flex;flex-
direction:column;min-height:100vh;position:sticky;top:0;height:100vh;overflow:auto}.brand{display:flex;align-
items:center;gap:11px;padding:2px 12px 28px;color:white;font-weight:800;letter-spacing:.13em;font-size:14px}.brand-mark{font-
size:25px;color:#37d3b6}.brand small{display:block;color:#8293a4;font-size:9px;letter-spacing:.18em;margin-top:3px;font-
weight:600}.workspace-label,.caption{color:#78899b;font-weight:700;letter-spacing:.17em;font-size:9px;padding:0 12px
9px}.workspace{display:flex;gap:10px;align-items:center;padding:10px;background:#142438;border:1px solid #26364a;border-radius:9px;margin-
bottom:23px;font-size:11px;font-weight:700}.workspace-badge{display:grid;place-
items:center;width:32px;height:32px;background:#234154;color:#69d8c0;border-radius:8px;font-size:10px}.workspace small,.profile
small{display:block;color:#8da0b1;font-weight:400;margin-top:3px}.caption{margin:9px 0 3px}.nav{display:grid;gap:3px}.nav
button{border:0;background:transparent;color:#a3b0be;display:flex;align-items:center;gap:11px;text-align:left;border-
radius:7px;padding:10px 12px;font-size:12px}.nav button:hover{background:#142337;color:white}.nav
button.active{background:#17382f;color:#6be0c4;font-weight:700}.nav .ico{width:17px;text-align:center;font-size:16px}.nav .count{margin-
left:auto;padding:2px 7px;border-radius:20px;background:#25374a;font-size:9px}.nav .count.alert{background:#432c34;color:#f29c91}.side-
bottom{margin-top:auto}.brief{padding:14px;margin:12px 4px 14px;border-radius:9px;background:linear-
gradient(145deg,#173b3a,#152c30);border:1px solid #294a48}.brief .spark{font-size:18px;color:#64d9bc}.brief strong{display:block;font-
size:12px;margin:8px 0 4px}.brief p{font-size:10px;color:#adc0bf;margin:0;line-height:1.55}.profile{display:flex;align-
items:center;gap:9px;padding:13px 7px 0;border-top:1px solid #263547;font-size:11px;font-weight:700}.avatar{display:grid;place-
items:center;width:31px;height:31px;background:#dcece9;color:#247d70;border-radius:50%;font-size:10px}.profile small{font-
size:10px}.main{min-width:0}.topbar{height:70px;background:#fff;border-bottom:1px solid var(--line);display:flex;align-
items:center;justify-content:space-between;padding:0 34px;gap:16px}.crumb{font-size:12px;color:#82909d}.crumb b{color:#18283a;margin-
left:10px}.top-actions{display:flex;align-items:center;gap:15px}.live{color:#258d78;font-size:10px;display:flex;align-
items:center;gap:7px;white-space:nowrap}.live i{width:7px;height:7px;background:#29bf99;border-radius:50%;box-shadow:0 0 0 4px
#e4f6f0}.btn{border:1px solid #e1e8eb;background:#fff;border-radius:6px;padding:8px 11px;color:#526273;font-size:11px}.btn:hover{border-
color:#98cfc2;color:#158a74}.btn.primary{background:#159d83;border-color:#159d83;color:white;font-weight:700}.top-
avatar{display:grid;place-items:center;width:32px;height:32px;border-radius:50%;background:#dcece9;color:#247d70;font-weight:700;font-
size:10px}.content{padding:30px 34px 42px;max-width:1550px;margin:auto}.eyebrow{font-size:9px;letter-spacing:.17em;font-
weight:800;color:#778897}.title-row{display:flex;justify-content:space-between;align-items:flex-end;gap:16px;margin-bottom:22px}.title-row
h1{font-size:25px;letter-spacing:-.04em;margin:5px 0 3px;color:#142437}.title-row p{font-size:11px;color:var(--
muted);margin:0}.actions{display:flex;gap:8px}.stats{display:grid;grid-template-columns:repeat(4,minmax(0,1fr));gap:13px;margin-
bottom:15px}.stat,.panel{background:#fff;border:1px solid var(--line);border-radius:10px;box-shadow:var(--shadow)}.stat{padding:16px
17px;min-height:112px}.stat-head{display:flex;justify-content:space-between;color:#607181;font-size:10px;font-weight:600}.stat-
icon{display:grid;place-items:center;width:27px;height:27px;border-radius:7px;background:#e8f6f2;color:#168c77;font-size:14px}.stat-
icon.blue{background:#eef2fc;color:#6079c1}.stat-icon.amber{background:#fbf2e4;color:#c18b3a}.stat-
icon.red{background:#fbebea;color:#c96f67}.stat-value{font-size:26px;font-weight:750;letter-spacing:-.05em;margin-top:8px}.stat-foot{font-
size:9px;color:#91a0aa;margin-top:1px}.overview-grid{display:grid;grid-template-columns:minmax(0,1.55fr)
minmax(280px,.9fr);gap:14px;margin-bottom:14px}.panel{padding:17px 18px}.panel-head{display:flex;align-items:flex-start;justify-
content:space-between;gap:12px;margin-bottom:13px}.panel-title{font-size:12px;font-weight:750;color:#26384a}.panel-sub{font-
size:9px;color:#929da6;margin-top:3px}.text-link{border:0;background:transparent;color:#159b83;font-size:10px;font-weight:700}.chart-
wrap{position:relative;width:100%;height:214px}.chart{display:block;width:100%;height:100%;overflow:visible}.chart
.gridline{stroke:#ebf0f1;stroke-width:1}.chart text{font:9px "Segoe UI",Arial,sans-serif;fill:#98a4ad}.chart
.line{fill:none;stroke:#19b69a;stroke-width:3;stroke-linecap:round;stroke-linejoin:round}.chart .area{fill:url(#area)}.chart
.dot{fill:white;stroke:#16a98d;stroke-width:2}.tooltip{position:absolute;z-index:3;display:none;pointer-
events:none;background:#132337;color:#fff;border-radius:7px;padding:8px 10px;min-width:135px;font-size:9px;line-height:1.5;box-shadow:0
8px 24px #13233735;transform:translate(-50%,-110%)}.tooltip.show{display:block}.tooltip b{display:block;font-size:11px;margin:2px
0}.status-list{display:grid;gap:0}.status-row{display:flex;align-items:center;gap:11px;padding:12px 0;border-bottom:1px solid
#eef1f2}.status-row:last-child{border-bottom:0}.status-symbol{display:grid;place-items:center;width:30px;height:30px;border-
radius:8px;background:#f0f5f6;color:#387c78;font-size:14px}.status-copy{flex:1}.status-copy strong{display:block;font-size:10px}.status-
copy small{font-size:9px;color:#919da6}.status-value{text-align:right;font-weight:750;font-size:14px}.status-value
small{display:block;font-size:8px;color:#9aa5ad;font-weight:400}.section-panel{margin-bottom:14px}.table-wrap{overflow:auto}table{border-
collapse:collapse;width:100%;min-width:650px}th{text-align:left;color:#8b98a2;text-transform:uppercase;font-size:8px;letter-
spacing:.08em;font-weight:700;padding:10px 9px;border-bottom:1px solid #e9eef0;white-space:nowrap}td{padding:11px 9px;border-bottom:1px
solid #f0f3f4;font-size:10px;color:#506071;white-space:nowrap}tbody tr{cursor:pointer}tbody tr:hover{background:#f7fbfa}td:first-
child{font-weight:700;color:#26394c}.badge{display:inline-flex;padding:4px 8px;border-radius:20px;font-size:8px;font-
weight:700;background:#edf2f3;color:#647480}.badge.green{background:#e6f5ee;color:#23886d}.badge.amber{background:#fbf1e0;color:#b37c28}.b
adge.red{background:#fbeae8;color:#bd5d57}.badge.blue{background:#edf1fb;color:#5f74b3}.badge.gray{background:#eef0f2;color:#687681}.lower
-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}.mini-row{display:flex;align-items:center;gap:10px;padding:10px 0;border-top:1px
solid #eef1f2}.mini-mark{display:grid;place-items:center;width:27px;height:27px;border-radius:7px;background:#fff0eb;color:#cb7564}.mini-
row strong{display:block;font-size:10px}.mini-row small{display:block;color:#909da6;font-size:9px;margin-top:3px}.mini-row .push{margin-
left:auto;color:#b96d54;font-size:9px;font-weight:700}.controls{display:flex;align-
items:center;gap:8px}.search{width:min(250px,42vw);border:1px solid #e4e9eb;background:#fff;border-radius:6px;padding:8px 10px;font-
size:10px;outline:none}.search:focus,select:focus{border-color:#21aa90;box-shadow:0 0 0 3px #e6f6f2}select{border:1px solid
#e4e9eb;background:#fff;border-radius:6px;padding:8px 10px;color:#607181;font-size:10px}.empty{text-
align:center;color:#83909b;padding:42px 20px;font-size:11px}.modal-bg{position:fixed;inset:0;display:none;align-items:center;justify-
content:center;padding:18px;background:#0b1421a8;z-index:20}.modal-bg.open{display:flex}.modal{position:relative;background:#fff;border-
radius:11px;width:min(540px,100%);max-height:88vh;overflow:auto;padding:24px;box-shadow:0 28px 90px #07111e55}.modal-
close{position:absolute;right:14px;top:10px;border:0;background:transparent;font-size:22px;color:#77838d}.modal h2{margin:6px 0;font-
size:20px}.modal p{font-size:11px;color:var(--muted);margin:0 0 16px}.detail-grid{display:grid;grid-template-columns:1fr
1fr;gap:1px;background:var(--line);border:1px solid var(--line);border-radius:7px;overflow:hidden}.detail-
cell{background:white;padding:11px}.detail-cell small{display:block;color:#8a969f;font-size:8px;margin-bottom:5px}.detail-cell
strong{font-size:10px}.form-grid{display:grid;grid-template-columns:1fr 1fr;gap:12px}.form-grid label{font-size:9px;color:#697986;font-
weight:700}.form-grid input,.form-grid select{display:block;width:100%;margin-top:5px;font-size:10px}.modal-actions{display:flex;justify-
content:flex-end;gap:8px;margin-top:18px}.toast{position:fixed;right:22px;bottom:22px;background:#142438;color:white;padding:11px
15px;border-radius:7px;font-size:10px;box-shadow:0 8px 30px #17263830;opacity:0;transform:translateY(12px);transition:.2s;pointer-
events:none;z-index:25}.toast.show{opacity:1;transform:translateY(0)}
```

```
0008  @media(max-width:1050px){.layout{grid-template-columns:220px 1fr}.content{padding:25px 22px}.topbar{padding:0 22px}.stats{grid-
template-columns:repeat(2,minmax(0,1fr))}}
0009  body.auth-required .layout{display:none}.auth-screen{display:none;min-height:100vh;place-
items:center;padding:24px;background:radial-gradient(ellipse at 18% 10%,#1a6052 0,transparent 38%),linear-gradient(135deg,#0b1421,#13283b
58%,#0e3c38);color:#e7eef3}.auth-required .auth-screen{display:grid}.auth-
box{width:min(430px,100%);padding:30px;background:#fff;color:var(--ink);border:1px solid #dbe5e8;border-radius:10px;box-shadow:0 28px 90px
#06111c70;animation:auth-in .35s ease-out}.auth-brand{display:flex;align-items:center;gap:10px;color:#15283b;font-weight:800;letter-
spacing:.12em}.auth-brand-mark{color:#159d83;font-size:25px}.auth-kicker{margin-top:29px;color:#159b83;font-size:9px;font-
weight:800;letter-spacing:.16em}.auth-box h1{margin:5px 0 5px;font-size:24px}.auth-copy{margin:0 0 22px;color:#7f8b98;font-
size:12px}.auth-form{display:grid;gap:13px}.auth-form label{display:grid;gap:5px;color:#61717e;font-size:10px;font-weight:700}.auth-form
input{width:100%;padding:10px 11px;border:1px solid #dce5e8;border-radius:6px;outline:none}.auth-form input:focus{border-
color:#21aa90;box-shadow:0 0 0 3px #e6f6f2}.auth-submit{width:100%;margin-top:4px}.auth-switch{margin:18px 0 0;text-
align:center;color:#788793;font-size:11px}.auth-switch button{border:0;background:none;color:#128b75;font-weight:700}.auth-
error{display:none;margin:0;padding:9px 10px;border-radius:6px;background:#fbeae8;color:#a84f49;font-size:11px}.auth-
error.show{display:block}.auth-note{margin-top:20px;padding-top:13px;border-top:1px solid #edf1f2;color:#98a3ab;font-size:9px;text-
align:center}@keyframes auth-in{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}@media(max-
width:500px){.auth-box{padding:23px}}
0010  @media(max-width:720px){.layout{grid-template-columns:66px 1fr}.sidebar{padding:18px 7px}.brand{padding:0 12px
25px}.brand>span:last-child,.workspace-label,.workspace,.caption,.nav-label,.brief,.profile>span,.profile>b{display:none}.nav
button{justify-content:center;padding:10px 0;font-size:0}.nav .ico{font-size:17px}.nav .count{display:none}.side-bottom{margin-
top:auto}.profile{justify-content:center;padding-left:0;padding-right:0}.topbar{height:60px;padding:0 13px}.crumb{font-size:10px}.top-
actions{gap:8px}.live{font-size:0}.live i{font-size:0}.top-actions .btn{font-size:0;padding:7px}.content{padding:19px 13px}.title-
row{align-items:flex-start;flex-direction:column}.title-row h1{font-size:22px}.actions{width:100%}.overview-grid,.lower-grid{grid-
template-columns:1fr}.stats{gap:8px}.stat{padding:12px;min-height:98px}.stat-value{font-size:22px}.panel{padding:14px}.chart-
wrap{height:180px}}
0011  </style>
0012  <style>.auth-form [hidden]{display:none}</style>
0013  </head>
0014  <body class="auth-required">
0015  <section class="auth-screen" id="authScreen" aria-label="Sign in">
0016    <div class="auth-box">
0017      <div class="auth-brand"><span class="auth-brand-mark">✳</span><span>NORTHSTAR</span></div>
0018      <div class="auth-kicker">FLIGHT OPERATIONS</div>
0019      <h1 id="authTitle">Welcome back</h1>
0020      <p class="auth-copy" id="authCopy">Sign in to access the airline operations workspace.</p>
0021      <form class="auth-form" id="authForm">
0022        <label id="authNameField" hidden>Name<input name="name" autocomplete="name" maxlength="80"></label>
0023        <label>Email<input name="email" type="email" autocomplete="email" required maxlength="254"></label>
0024        <label>Password<input name="password" type="password" autocomplete="current-password" required minlength="10"></label>
0025        <p class="auth-error" id="authError" role="alert"></p>
0026        <button class="btn primary auth-submit" id="authSubmit" type="submit">Sign in</button>
0027      </form>
0028      <p class="auth-switch"><span id="authSwitchText">New to Northstar?</span> <button id="authModeButton" type="button">Create an
account</button></p>
0029      <div class="auth-note">Local airline operations workspace</div>
0030    </div>
0031  </section>
0032  <div class="layout">
0033    <aside class="sidebar">
0034      <div class="brand"><span class="brand-mark">✳</span><span>NORTHSTAR<small>FLIGHT OPERATIONS</small></span></div>
0035      <div class="workspace-label">WORKSPACE</div><div class="workspace"><span class="workspace-badge">NS</span><span>Northstar
Airways<small>Airline logistics demo</small></span></div>
0036      <div class="caption">OPERATIONS</div><nav class="nav" id="nav">
0037        <button class="active" data-page="overview"><span class="ico">◫</span><span class="nav-label">Overview</span></button>
0038        <button data-page="flights"><span class="ico">↗</span><span class="nav-label">Flight schedule</span><span class="count"
id="flightCount">12</span></button>
0039        <button data-page="fleet"><span class="ico">✈</span><span class="nav-label">Fleet</span></button>
0040        <button data-page="crew"><span class="ico">♙</span><span class="nav-label">Crew roster</span></button>
0041        <button data-page="cargo"><span class="ico">▦</span><span class="nav-label">Cargo tracking</span></button>
0042        <button data-page="passengers"><span class="ico">♔</span><span class="nav-label">Passengers & loyalty</span></button>
0043      </nav>
0044      <div class="caption">NETWORK WATCH</div><nav class="nav"><button data-page="disruptions"><span class="ico">⌁</span><span
class="nav-label">Disruptions</span><span class="count alert" id="disruptionCount">2</span></button></nav>
0045      <div class="caption">SUSTAINABILITY</div><nav class="nav"><button data-page="emissions"><span class="ico">◉</span><span
class="nav-label">Emissions</span></button></nav>
0046      <div class="side-bottom"><div class="brief"><span class="spark">✦</span><strong>Operations brief</strong><p>Review the network
snapshot and sample operational alerts.</p></div><div class="profile"><div class="avatar" id="profileInitials">NS</div><span
id="profileName">Account<small>Signed in</small></span><button class="btn" id="signOutBtn" type="button" title="Sign
out">↪</button></div></div>
0047    </aside>
0048    <main class="main">
0049      <header class="topbar"><div class="crumb">Northstar Airways <span> / </span><b id="crumb">Overview</b></div><div class="top-
actions"><span class="live" id="connectionStatus"><i></i> CONNECTING</span><button class="btn" id="resetBtn" title="Refresh from
Oracle">↻ Refresh</button><button class="btn" id="notifyBtn" aria-label="Notifications">♤</button><div class="top-avatar"
id="topAvatar">NS</div></div></header>
0050      <section class="content" id="content"></section>
0051    </main>
0052  </div>
0053  <div class="modal-bg" id="modalBg"><section class="modal"><button class="modal-close" id="modalClose">×</button><div
id="modalBody"></div></section></div><div class="toast" id="toast"></div>
0054  <script>
0055  const seed={
0056   flights:[
0057    {id:'NS 204',route:'DEL → DXB',departure:'09:40',arrival:'12:15',aircraft:'VT-NXA · A321neo',gate:'A12',status:'Boarding'},
0058    {id:'NS 118',route:'BOM → SIN',departure:'10:25',arrival:'18:05',aircraft:'VT-NXB · B787-9',gate:'B06',status:'On time'},
0059    {id:'NS 356',route:'BLR → LHR',departure:'11:10',arrival:'17:45',aircraft:'VT-NXC · A350-900',gate:'C03',status:'On time'},
0060    {id:'NS 087',route:'DEL → LHR',departure:'12:30',arrival:'18:00',aircraft:'VT-NXD · B787-9',gate:'A08',status:'Delayed'},
0061    {id:'NS 412',route:'HYD → FRA',departure:'13:05',arrival:'19:10',aircraft:'VT-NXE · A321neo',gate:'D02',status:'On time'},
0062    {id:'NS 229',route:'MAA → CMB',departure:'14:20',arrival:'15:45',aircraft:'VT-NXF · A320',gate:'B14',status:'Boarding'},
```

```
0063    {id:'NS 630',route:'DEL → BOM',departure:'15:00',arrival:'17:10',aircraft:'VT-NXG · A320neo',gate:'A04',status:'On time'},
0064    {id:'NS 504',route:'CCU → SIN',departure:'16:40',arrival:'23:00',aircraft:'VT-NXH · A321neo',gate:'C08',status:'Scheduled'},
0065    {id:'NS 777',route:'BLR → DXB',departure:'18:15',arrival:'21:05',aircraft:'VT-NXI · A321neo',gate:'B09',status:'Scheduled'},
0066    {id:'NS 291',route:'BOM → DEL',departure:'19:30',arrival:'21:40',aircraft:'VT-NXJ · A320',gate:'A16',status:'Scheduled'},
0067    {id:'NS 642',route:'DEL → SIN',departure:'21:00',arrival:'05:25',aircraft:'VT-NXB · B787-9',gate:'A10',status:'Scheduled'},
0068    {id:'NS 094',route:'HYD → BOM',departure:'22:15',arrival:'23:50',aircraft:'VT-NXK · A320neo',gate:'D07',status:'Scheduled'}
0069   ],
0070   fleet:[
0071    {id:'VT-NXA',model:'Airbus A321neo',capacity:'24,000 kg',efficiency:'2.8 L / 100 km',status:'Active',flights:'NS 204'},
0072    {id:'VT-NXB',model:'Boeing 787-9',capacity:'44,000 kg',efficiency:'2.3 L / 100 km',status:'Active',flights:'NS 118 · NS 642'},
0073    {id:'VT-NXC',model:'Airbus A350-900',capacity:'45,000 kg',efficiency:'2.5 L / 100 km',status:'Active',flights:'NS 356'},
0074    {id:'VT-NXD',model:'Boeing 787-9',capacity:'44,000 kg',efficiency:'2.3 L / 100 km',status:'Active',flights:'NS 087'},
0075    {id:'VT-NXE',model:'Airbus A321neo',capacity:'24,000 kg',efficiency:'2.8 L / 100 km',status:'Active',flights:'NS 412'},
0076    {id:'VT-NXF',model:'Airbus A320',capacity:'19,000 kg',efficiency:'3.1 L / 100 km',status:'Maintenance',flights:'—'},
0077    {id:'VT-NXG',model:'Airbus A320neo',capacity:'20,000 kg',efficiency:'2.9 L / 100 km',status:'Active',flights:'NS 630'},
0078    {id:'VT-NXH',model:'Airbus A321neo',capacity:'24,000 kg',efficiency:'2.8 L / 100 km',status:'Active',flights:'NS 504'}
0079   ],
0080   crew:[
0081    {id:'CR-021',name:'Aarav Mehta',role:'Captain',base:'Delhi',flight:'NS 204',shift:'6.2 h',status:'On duty'},
0082    {id:'CR-034',name:'Mira Shah',role:'First Officer',base:'Mumbai',flight:'NS 118',shift:'5.8 h',status:'On duty'},
0083    {id:'CR-048',name:'Kabir Rao',role:'Cabin lead',base:'Bengaluru',flight:'NS 356',shift:'7.1 h',status:'On duty'},
0084    {id:'CR-055',name:'Anaya Iyer',role:'Captain',base:'Delhi',flight:'NS 087',shift:'4.4 h',status:'On duty'},
0085    {id:'CR-069',name:'Ishaan Das',role:'First Officer',base:'Hyderabad',flight:'NS 412',shift:'5.1 h',status:'On duty'},
0086    {id:'CR-073',name:'Sana Kapoor',role:'Cabin crew',base:'Chennai',flight:'NS 229',shift:'3.5 h',status:'Standby'}
0087   ],
0088   cargo:[
0089    {id:'CG-7401',name:'Temperature-controlled biologics',category:'Healthcare',flight:'NS 204',weight:'820 kg',required:'2–8
°C',current:'4.2 °C',tracking:'On track'},
0090    {id:'CG-7402',name:'Fresh seafood consignment',category:'Perishable',flight:'NS 118',weight:'1,260 kg',required:'0–4
°C',current:'3.1 °C',tracking:'On track'},
0091    {id:'CG-7403',name:'Precision instrument cases',category:'High value',flight:'NS 356',weight:'445
kg',required:'Ambient',current:'22 °C',tracking:'On track'},
0092    {id:'CG-7404',name:'Clinical trial samples',category:'Healthcare',flight:'NS 087',weight:'310 kg',required:'2–8 °C',current:'9.1
°C',tracking:'Alert'},
0093    {id:'CG-7405',name:'Specialty produce',category:'Perishable',flight:'NS 412',weight:'970 kg',required:'4–8 °C',current:'6.3
°C',tracking:'On track'}
0094   ],
0095   passengers:[
0096    {id:'P-28104',name:'Aditi Sharma',tier:'Summit',allowance:'+15 kg',baggage:'28 kg',flight:'NS 204'},
0097    {id:'P-28105',name:'Rohan Nair',tier:'Aurora',allowance:'+5 kg',baggage:'22 kg',flight:'NS 118'},
0098    {id:'P-28106',name:'Meera Kulkarni',tier:'Horizon',allowance:'+0 kg',baggage:'18 kg',flight:'NS 356'},
0099    {id:'P-28107',name:'Arjun Patel',tier:'Summit',allowance:'+15 kg',baggage:'31 kg',flight:'NS 087'},
0100    {id:'P-28108',name:'Diya Bose',tier:'Aurora',allowance:'+5 kg',baggage:'20 kg',flight:'NS 412'},
0101    {id:'P-28109',name:'Neel Menon',tier:'Horizon',allowance:'+0 kg',baggage:'17 kg',flight:'NS 229'}
0102   ],
0103   disruptions:[
0104    {id:'DP-5101',flight:'NS 087',route:'DEL → LHR',reason:'Weather restriction',delay:'42 min',date:'10 Oct ·
12:30',status:'Monitoring'},
0105    {id:'DP-5102',flight:'NS 412',route:'HYD → FRA',reason:'Ground handling',delay:'18 min',date:'10 Oct · 13:05',status:'Response
active'},
0106    {id:'DP-5103',flight:'NS 356',route:'BLR → LHR',reason:'Air traffic flow',delay:'12 min',date:'10 Oct · 11:10',status:'Resolved'}
0107   ],
0108   emissions:[
0109    {id:'EM-8801',flight:'NS 204',route:'DEL → DXB',fuel:'6,840 L',co2:'18.2 t',aircraft:'VT-NXA',logged:'10 Oct · 12:25'},
0110    {id:'EM-8802',flight:'NS 118',route:'BOM → SIN',fuel:'14,220 L',co2:'37.8 t',aircraft:'VT-NXB',logged:'10 Oct · 18:10'},
0111    {id:'EM-8803',flight:'NS 356',route:'BLR → LHR',fuel:'22,680 L',co2:'60.3 t',aircraft:'VT-NXC',logged:'10 Oct · 17:52'},
0112    {id:'EM-8804',flight:'NS 087',route:'DEL → LHR',fuel:'19,240 L',co2:'51.1 t',aircraft:'VT-NXD',logged:'10 Oct · 18:12'},
0113    {id:'EM-8805',flight:'NS 412',route:'HYD → FRA',fuel:'16,400 L',co2:'43.5 t',aircraft:'VT-NXE',logged:'10 Oct · 19:20'},
0114    {id:'EM-8806',flight:'NS 229',route:'MAA → CMB',fuel:'4,620 L',co2:'12.3 t',aircraft:'VT-NXF',logged:'10 Oct · 15:55'}
0115   ]
0116  };
0117  let
data={flights:[],fleet:[],crew:[],cargo:[],passengers:[],disruptions:[],emissions:[],loyalty_tiers:[]},page='overview',toastTimer,firstRen
der=true,deferredRender=false;const configs={
0118   flights:{title:'Flight schedule',sub:'Coordinate departures, aircraft and crew across the
network.',icon:'↗',cols:[['Flight','id'],['Route','route'],['Departure','departure'],['Arrival','arrival'],['Aircraft','aircraft'],['Statu
s','status']]},
0119   fleet:{title:'Fleet',sub:'Aircraft capability and availability across the network.',icon:'✈',cols:[['Aircraft ID','id'],['Aircraft
model','model'],['Capacity','capacity'],['Fuel efficiency','efficiency'],['Today’s flight','flights'],['Status','status']]},
0120   crew:{title:'Crew roster',sub:'Assignments, duty hours and operating bases.',icon:'♙',cols:[['Crew
member','name'],['Role','role'],['Base','base'],['Flight assignment','flight'],['Shift hours','shift'],['Status','status']]},
0121   cargo:{title:'Cargo tracking',sub:'Special cargo manifests and temperature
checkpoints.',icon:'▦',cols:[['Shipment','name'],['Category','category'],['Flight','flight'],['Weight','weight'],['Required
temp.','required'],['Current temp.','current'],['Tracking','tracking']]},
0122   passengers:{title:'Passengers & loyalty',sub:'Passenger baggage details and loyalty tier
allowances.',icon:'♔',cols:[['Passenger','name'],['Passenger ID','id'],['Loyalty
tier','tier'],['Allowance','allowance'],['Baggage','baggage'],['Flight','flight']]},
0123   disruptions:{title:'Disruptions',sub:'Delay events, reason codes and response status linked to
flights.',icon:'⌁',cols:[['Event','id'],['Flight','flight'],['Route','route'],['Reason','reason'],['Delay','delay'],['Event
time','date'],['Status','status']]},
0124   emissions:{title:'Emissions',sub:'Flight-level fuel and CO₂ logs for sustainability
reporting.',icon:'◉',cols:[['Log','id'],['Flight','flight'],['Route','route'],['Fuel
logged','fuel'],['CO₂','co2'],['Aircraft','aircraft'],['Logged at','logged']]}
0125  };
0126  const content=document.getElementById('content'),crumb=document.getElementById('crumb');function clone(v){return
JSON.parse(JSON.stringify(v))}function esc(v){return
String(v??'').replace(/[&<>"']/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[c]))}function toast(msg){let
t=document.getElementById('toast');t.textContent=msg;t.classList.add('show');clearTimeout(toastTimer);toastTimer=setTimeout(()=>t.classLis
t.remove('show'),2400)}function badge(v){let
s=String(v||'').toLowerCase(),c=s.includes('alert')||s.includes('delay')||s.includes('monitor')?'red':s.includes('board')||s.includes('res
```

```
ponse')||s.includes('maint')?'amber':s.includes('active')||s.includes('track')||s.includes('resolved')||s.includes('duty')?'green':s.inclu
des('scheduled')?'blue':'gray';return `<span class="badge ${c}">${esc(v)}</span>`}function table(rows,cols){if(!rows.length)return '<div
class="empty">No records match your search.</div>';return `<div class="table-
wrap"><table><thead><tr>${cols.map(c=>`<th>${esc(c[0])}</th>`).join('')}</tr></thead><tbody>${rows.map((r,i)=>`<tr data-
row="${i}">${cols.map((c,j)=>`<td>${c[1]==='status'||c[1]==='tracking'?badge(r[c[1]]):esc(r[c[1]])}</td>`).join('')}</tr>`).join('')}</tbo
dy></table></div>`}
0127  function titleBlock(title,sub,actions=''){return `<div class="title-row"><div><div class="eyebrow">NORTHSTAR AIRWAYS ·
${page==='overview'?'NETWORK CONTROL':'OPERATIONS CONSOLE'}</div><h1>${esc(title)}</h1><p>${esc(sub)}</p></div><div
class="actions">${actions}</div></div>`}function panelHead(title,sub,link=''){return `<div class="panel-head"><div><div class="panel-
title">${title}</div><div class="panel-sub">${sub}</div></div>${link}</div>`}
0128  function stats(){let co2=data.emissions.reduce((n,r)=>n+parseFloat(r.co2),0);return `<div class="stats"><article class="stat"><div
class="stat-head">Flights today <span class="stat-icon">↗</span></div><div class="stat-value">${data.flights.length}</div><div
class="stat-foot">Across ${data.flights.length} flights in Oracle</div></article><article class="stat"><div class="stat-head">Fleet in
service <span class="stat-icon blue">✈</span></div><div class="stat-
value">${data.fleet.filter(x=>String(x.status).toLowerCase()==='active').length} <small style="font-size:13px;color:#9aa5ad">/
${data.fleet.length}</small></div><div class="stat-foot">Aircraft available today</div></article><article class="stat"><div class="stat-
head">Open disruptions <span class="stat-icon amber">⌁</span></div><div class="stat-
value">${data.disruptions.filter(x=>!String(x.status).toLowerCase().includes('resolved')).length}</div><div class="stat-foot">Events
requiring coordination</div></article><article class="stat"><div class="stat-head">CO₂ logged <span class="stat-icon
red">◉</span></div><div class="stat-value">${co2.toFixed(1)} <small style="font-size:13px;color:#9aa5ad">t</small></div><div class="stat-
foot">From ${data.emissions.length} Oracle flight logs</div></article></div>`}
0129  function chart(){let vals=[3,5,4,7,6,9,8,11,9],xs=vals.map((_,i)=>54+i*88),ys=vals.map(v=>170-
v*11),path=xs.map((x,i)=>`${i?'L':'M'}${x} ${ys[i]}`).join(' '),area=`${path} L758 180 L54 180 Z`;return `<div class="chart-wrap"
id="chartWrap"><svg class="chart" viewBox="0 0 800 220" preserveAspectRatio="none" aria-label="Flight departures by
time"><defs><linearGradient id="area" x1="0" x2="0" y1="0" y2="1"><stop offset="0" stop-color="#17b394" stop-opacity=".23"/><stop
offset="1" stop-color="#17b394" stop-opacity="0"/></linearGradient></defs>${[30,70,110,150,190].map(y=>`<line class="gridline" x1="48"
y1="${y}" x2="780" y2="${y}"/>`).join('')}${['06:00','09:00','12:00','15:00','18:00'].map((x,i)=>`<text x="${54+i*176}"
y="211">${x}</text>`).join('')}<path class="area" d="${area}"/><path class="line" d="${path}"/>${xs.map((x,i)=>`<circle class="dot"
cx="${x}" cy="${ys[i]}" r="4"/>`).join('')}</svg><div class="tooltip" id="chartTip"></div></div>`}
0130  function overview(){return `${titleBlock('Good morning, team','Here's the latest view across your airline network.',`<button
class="btn" data-export="flights">↓ Export report</button><button class="btn primary" data-add="flights">+ New
flight</button>`)}${stats()}<div class="overview-grid"><section class="panel">${panelHead('Departure flow','Sample departures · local
time','<button class="text-link" data-page="flights">View schedule →</button>')}${chart()}</section><section
class="panel">${panelHead('Live operations','Network status at a glance')}<div class="status-list"><div class="status-row"><span
class="status-symbol">✈</span><span class="status-copy"><strong>Flights scheduled</strong><small>Across today's
network</small></span><span class="status-value">${data.flights.length}<small>flights</small></span></div><div class="status-row"><span
class="status-symbol">♙</span><span class="status-copy"><strong>Crew on duty</strong><small>Assigned across the
network</small></span><span class="status-value">${data.crew.length}<small>members</small></span></div><div class="status-row"><span
class="status-symbol">⌁</span><span class="status-copy"><strong>Open disruptions</strong><small>Events still being
monitored</small></span><span class="status-
value">${data.disruptions.filter(x=>!x.status.toLowerCase().includes('resolved')).length}<small>events</small></span></div></div></section
></div><section class="panel section-panel">${panelHead('Upcoming departures','Next scheduled flights','<button class="text-link" data-
page="flights">All flights →</button>')}${table(data.flights.slice(0,5),configs.flights.cols)}</section><div class="lower-grid"><section
class="panel">${panelHead('Cold-chain watch','Cargo temperature checkpoints','<button class="text-link" data-page="cargo">View cargo
→</button>')}${data.cargo.slice(0,3).map(c=>`<div class="mini-row"><span class="status-symbol">▦</span><span
style="flex:1"><strong>${esc(c.name)}</strong><small>${esc(c.id)} · ${esc(c.flight)} ·
${esc(c.current)}</small></span>${badge(c.tracking)}</div>`).join('')}</section><section class="panel">${panelHead('Disruption
desk','Events needing coordination','<button class="text-link" data-page="disruptions">View all
→</button>')}${data.disruptions.slice(0,3).map(d=>`<div class="mini-row"><span class="mini-mark">!</span><span
style="flex:1"><strong>${esc(d.flight)} · ${esc(d.reason)}</strong><small>${esc(d.route)} · ${esc(d.date)}</small></span><span
class="push">+${esc(d.delay)}</span></div>`).join('')}</section></div>`}
0131  function section(view){let c=configs[view],rows=data[view],statuses=[...new
Set(rows.map(r=>r.status||r.tracking).filter(Boolean))];return `${titleBlock(c.title,c.sub,`<button class="btn" data-export="${view}">↓
Export CSV</button><button class="btn primary" data-add="${view}">+ Add
${view==='flights'?'flight':view==='fleet'?'aircraft':view==='crew'?'crew
member':view==='cargo'?'shipment':view==='passengers'?'passenger':view==='emissions'?'log':'event'}</button>`)}${view==='emissions'?`<div
class="stats">${emissionStats()}</div>`:''}<section class="panel section-panel">${panelHead(`<span style="color:#159b83">${c.icon}</span>
 ${c.title}`,`${rows.length} records · Sample data`,`<div class="controls"><input class="search" id="search" placeholder="⌕  Search
${view}…" type="search"><select id="filter"><option value="">All
statuses</option>${statuses.map(s=>`<option>${esc(s)}</option>`).join('')}</select></div>`)}<div
id="tableArea">${table(rows,c.cols)}</div></section>${view==='passengers'?`<section class="panel">${panelHead('Loyalty tier
allowances','Sample tier and baggage allowance reference')}<div class="status-list"><div class="status-row"><span class="status-
symbol">✦</span><span class="status-copy"><strong>Summit</strong><small>Additional baggage allowance</small></span><span class="status-
value">+15 kg</span></div><div class="status-row"><span class="status-symbol">✦</span><span class="status-
copy"><strong>Aurora</strong><small>Additional baggage allowance</small></span><span class="status-value">+5 kg</span></div><div
class="status-row"><span class="status-symbol">✦</span><span class="status-copy"><strong>Horizon</strong><small>Additional baggage
allowance</small></span><span class="status-value">+0 kg</span></div></div></section>`:''}`}
0132  function emissionStats(){let fuel=data.emissions.reduce((n,r)=>n+Number(r.fuel.replace(/[
,L]/g,'')),0),co2=data.emissions.reduce((n,r)=>n+Number(r.co2.replace(/[ t]/g,'')),0);return `<article class="stat"><div class="stat-
head">CO₂ logged <span class="stat-icon red">◉</span></div><div class="stat-value">${co2.toFixed(1)} t</div><div class="stat-foot">Across
sample flight logs</div></article><article class="stat"><div class="stat-head">Emission records <span class="stat-icon
blue">▤</span></div><div class="stat-value">${data.emissions.length}</div><div class="stat-foot">Illustrative
dataset</div></article><article class="stat"><div class="stat-head">Fuel logged <span class="stat-icon amber">○</span></div><div
class="stat-value">${fuel.toLocaleString()}</div><div class="stat-foot">Liters across records</div></article><article class="stat"><div
class="stat-head">Average per log <span class="stat-icon">↘</span></div><div class="stat-value">${(co2/data.emissions.length).toFixed(1)}
t</div><div class="stat-foot">Sample CO₂ per flight</div></article>`}
0133  function render(view='overview'){page=view;document.querySelectorAll('[data-
page]').forEach(b=>b.classList.toggle('active',b.dataset.page===view));crumb.textContent=view==='overview'?'Overview':configs[view].title;
content.innerHTML=view==='overview'?overview():section(view);if(view==='overview'&&data.flights.length)bindChart();if(view!=='overview')bi
ndFilters(view)}
0134  function bindChart(){let
wrap=document.getElementById('chartWrap'),tip=document.getElementById('chartTip'),svg=wrap.querySelector('svg');svg.addEventListener('poin
termove',e=>{let r=svg.getBoundingClientRect(),ratio=Math.max(0,Math.min(1,(e.clientX-r.left)/r.width)),idx=Math.min(data.flights.length-
1,Math.round(ratio*(data.flights.length-
1))),f=data.flights[idx];if(!f)return;tip.style.left=`${Math.max(12,Math.min(88,ratio*100))}%`;tip.style.top=`${Math.max(8,e.clientY-
r.top-8)}px`;tip.innerHTML=`Flight activity<b>${esc(f.id)} · ${esc(f.departure)}</b>${esc(f.route)} ·
${esc(f.status)}`;tip.classList.add('show')});svg.addEventListener('pointerleave',()=>tip.classList.remove('show'))}
0135  function activeRows(view){let
q=(document.getElementById('search')?.value||'').toLowerCase(),status=document.getElementById('filter')?.value||'';return
```

```
data[view].filter(r=>JSON.stringify(r).toLowerCase().includes(q)&&(!status||(r.status||r.tracking)===status))}function
bindFilters(view){document.getElementById('search').addEventListener('input',()=>updateTable(view));document.getElementById('filter').addE
ventListener('change',()=>updateTable(view))}function
updateTable(view){document.getElementById('tableArea').innerHTML=table(activeRows(view),configs[view].cols)}
0136  const formFields={
0137    flights:[['route','Route','text'],['departure','Departure','datetime-local'],['arrival','Arrival','datetime-
local'],['aircraft_id','Aircraft','fleet'],['status','Status','text']],
0138    fleet:[['model','Aircraft model','text'],['capacity_kg','Capacity (kg)','number'],['efficiency','Fuel
efficiency','number'],['status','Status','text']],
0139    crew:[['name','Crew member','text'],['role','Role','text'],['base','Base location','text']],
0140    cargo:[['flight_id','Flight','flights'],['name','Description','text'],['weight_kg','Weight
(kg)','number'],['category','Category','text'],['required_temp_c','Required temperature (°C)','number'],['current_temp_c','Current
temperature (°C)','number'],['tracking','Tracking status','text']],
0141    passengers:[['name','Passenger name','text'],['passport_no','Passport number','text'],['tier_id','Loyalty
tier','tiers'],['baggage_kg','Baggage (kg)','number']],
0142    disruptions:[['flight_id','Flight','flights'],['delay_min','Delay (minutes)','number'],['reason','Reason
code','text'],['event_date','Event time','datetime-local'],['status','Status','text']],
0143    emissions:[['flight_id','Flight','flights'],['fuel_liters','Fuel (liters)','number'],['co2_tons','CO₂
(tons)','number'],['logged_at','Logged at','datetime-local']]
0144  };
0145  function inputFor([key,label,type],record={}){let
value=record[key]??(key==='departure'?record.departure_iso:key==='arrival'?record.arrival_iso:'')??'',required=key!=='db_id'&&key!=='requi
red_temp_c'&&key!=='current_temp_c';if(type==='fleet'||type==='flights'||type==='tiers'){let
opts=type==='fleet'?data.fleet.map(x=>[x.db_id,x.id+' · '+x.model]):type==='flights'?data.flights.map(x=>[x.db_id,x.id+' ·
'+x.route]):data.loyalty_tiers.map(x=>[x.tier_id,x.tier_name]);return `<label>${esc(label)}<select name="${esc(key)}"
${required?'required':''}><option value="">Choose…</option>${opts.map(([id,text])=>`<option value="${esc(id)}"
${String(value)===String(id)?'selected':''}>${esc(text)}</option>`).join('')}</select></label>`}if(type==='datetime-
local'&&value)value=String(value).slice(0,16);return `<label>${esc(label)}<input name="${esc(key)}" type="${type}" value="${esc(value)}"
${type==='number'?'step="any"':''} ${required?'required':''}></label>`}
0146  async function openRecordForm(view,row=null){let fields=row?formFields[view]:[['db_id','Numeric ID (leave blank if Oracle generates
it)','number'],...formFields[view]];try{await Promise.all(['flights','fleet','tiers'].map(async reference=>{data[`ref_${reference}`]=await
apiRequest(`/api/reference/${reference}`)}))}catch(error){toast(error.message);return}document.getElementById('modalBody').innerHTML=`<div
class="eyebrow">${esc(configs[view].title.toUpperCase())} · ORACLE RECORD</div><h2>${row?'Edit':'Add'}
${esc(configs[view].title.replace(/s$/,''))}</h2><form id="recordForm"><div class="form-
grid">${fields.map(field=>inputFor(field,row||{})).join('')}</div><div class="modal-actions"><button type="button" class="btn"
id="cancelRecord">Cancel</button><button class="btn primary" type="submit">${row?'Save changes':'Add
record'}</button></div></form>`;document.getElementById('modalBg').classList.add('open');document.getElementById('cancelRecord').onclick=c
loseModal;document.getElementById('recordForm').onsubmit=async e=>{e.preventDefault();let payload={};for(const [key,value] of new
FormData(e.currentTarget).entries()){if(row&&key==='db_id')continue;const
field=fields.find(item=>item[0]===key);payload[key]=value===''?null:field&&(field[2]==='number'||field[2]==='fleet'||field[2]==='flights'|
|field[2]==='tiers')?Number(value):value}try{await
apiRequest(`/api/${view}${row?`/${row.db_id}`:''}`,row?'PUT':'POST',payload);closeModal();await refreshData(true);toast(row?'Oracle record
updated.':'Oracle record added.')}catch(error){toast(error.message)}}}
0147  function showDetail(view,row){let fields=configs[view].cols;document.getElementById('modalBody').innerHTML=`<div
class="eyebrow">${esc(configs[view].title.toUpperCase())} · ORACLE RECORD</div><h2>${esc(row.name||row.id||row.flight||'Record
detail')}</h2><p>${esc(row.route||row.model||row.role||row.category||row.reason||'Database record')}</p><div class="detail-
grid">${fields.map(([label,key])=>`<div class="detail-
cell"><small>${esc(label)}</small><strong>${key==='status'||key==='tracking'?badge(row[key]):esc(row[key])}</strong></div>`).join('')}</di
v><div class="modal-actions"><button class="btn" id="deleteRecord">Delete</button><button class="btn" id="editRecord">Edit</button><button
class="btn primary"
id="closeDetail">Done</button></div>`;document.getElementById('modalBg').classList.add('open');document.getElementById('closeDetail').oncl
ick=closeModal;document.getElementById('editRecord').onclick=()=>openRecordForm(view,row);document.getElementById('deleteRecord').onclick=
async()=>{if(!confirm(`Delete ${row.id||row.name||'this record'} from Oracle?`))return;try{await
apiRequest(`/api/${view}/${row.db_id}`,'DELETE');closeModal();await refreshData(true);toast('Oracle record
deleted.')}catch(error){toast(error.message)}}}
0148  function
closeModal(){document.getElementById('modalBg').classList.remove('open');if(deferredRender){deferredRender=false;render(page)}}
0149  function exportCsv(view){let
cols=configs[view].cols,rows=data[view],csv=[cols.map(x=>x[0]).join(','),...rows.map(r=>cols.map(([,k])=>`"${String(r[k]??'').replaceAll('
"','""')}"`).join(','))].join('\n'),a=document.createElement('a');a.href=URL.createObjectURL(new
Blob([csv],{type:'text/csv'}));a.download=`northstar-${view}.csv`;a.click();URL.revokeObjectURL(a.href);toast('CSV downloaded.')}
0150  function addRecord(view){openRecordForm(view)}
0151  async function apiRequest(url,method='GET',body){let response;try{response=await fetch(url,{method,headers:body?{'Content-
Type':'application/json'}:undefined,body:body?JSON.stringify(body):undefined})}catch{throw new Error('Cannot reach the API. Start the
server and open http://127.0.0.1:8001.')}let result=await response.json().catch(()=>({}));if(!response.ok)throw new
Error(result.detail||`Request failed (${response.status}).`);return result}
0152  async function refreshData(force=false){let next=await
apiRequest('/api/data'),changed=JSON.stringify(next)!==JSON.stringify(data);data=next;document.getElementById('connectionStatus').innerHTM
L='<i></i> ORACLE
CONNECTED';document.getElementById('connectionStatus').style.color='#258d78';document.getElementById('flightCount').textContent=data.fligh
ts.length;document.getElementById('disruptionCount').textContent=data.disruptions.filter(item=>!String(item.status).toLowerCase().includes
('resolved')).length;if(changed||force){if(document.getElementById('modalBg').classList.contains('open'))deferredRender=true;else
render(page)}return changed}
0153  async function startApp(){try{await refreshData(true)}catch(error){document.getElementById('connectionStatus').innerHTML='<i></i> DB
OFFLINE';document.getElementById('connectionStatus').style.color='#bd5d57';render('overview');toast(error.message)}finally{firstRender=fal
se}}
0154  document.addEventListener('click',e=>{let pageButton=e.target.closest('[data-
page]');if(pageButton){render(pageButton.dataset.page);return}let row=e.target.closest('tbody tr');if(row){let
view=page==='overview'?'flights':page,rows=page==='overview'?data.flights.slice(0,5):activeRows(view);if(rows[Number(row.dataset.row)])sho
wDetail(view,rows[Number(row.dataset.row)]);return}let add=e.target.closest('[data-add]');if(add){addRecord(add.dataset.add);return}let
ex=e.target.closest('[data-
export]');if(ex){exportCsv(ex.dataset.export);return}});document.getElementById('modalClose').onclick=closeModal;document.getElementById('
modalBg').addEventListener('click',e=>{if(e.target.id==='modalBg')closeModal()});document.addEventListener('keydown',e=>{if(e.key==='Escap
e')closeModal()});document.getElementById('resetBtn').onclick=()=>{data=clone(seed);render(page);toast('Sample data
reset.')};document.getElementById('notifyBtn').onclick=()=>toast('You're all caught up on
notifications.');document.getElementById('flightCount').textContent=seed.flights.length;document.getElementById('disruptionCount').textCon
tent=seed.disruptions.filter(x=>!x.status.toLowerCase().includes('resolved')).length;render('overview');
0155  </script>
0156  <script>
0157  let authUser=null,authMode='login';
```

```
0158  function setAuthMode(mode){authMode=mode;let signup=mode==='signup';document.getElementById('authTitle').textContent=signup?'Create
your account':'Welcome back';document.getElementById('authCopy').textContent=signup?'Create a local account to access flight
operations.':'Sign in to access the airline operations
workspace.';document.getElementById('authNameField').hidden=!signup;document.querySelector('#authNameField
input').required=signup;document.getElementById('authSubmit').textContent=signup?'Create account':'Sign
in';document.getElementById('authSwitchText').textContent=signup?'Already have an account?':'New to
Northstar?';document.getElementById('authModeButton').textContent=signup?'Sign in':'Create an account';document.querySelector('#authForm
[name="password"]').autocomplete=signup?'new-password':'current-password';document.getElementById('authError').classList.remove('show')}
0159  function showAuth(message=''){authUser=null;document.body.classList.add('auth-required');let
error=document.getElementById('authError');error.textContent=message;error.classList.toggle('show',Boolean(message));document.querySelecto
r('#authForm [name="password"]').value=''}
0160  function setSignedInUser(user){authUser=user;document.body.classList.remove('auth-
required');document.getElementById('profileName').innerHTML=`${esc(user.name)}<small>${esc(user.email)}</small>`;let
initials=user.name.trim().split(/\s+/).slice(0,2).map(part=>part[0]).join('').toUpperCase();document.getElementById('profileInitials').tex
tContent=initials;document.getElementById('topAvatar').textContent=initials}
0161  document.getElementById('authModeButton').onclick=()=>setAuthMode(authMode==='login'?'signup':'login');
0162  document.getElementById('authForm').onsubmit=async event=>{event.preventDefault();let form=new
FormData(event.currentTarget),payload={email:form.get('email'),password:form.get('password')};if(authMode==='signup')payload.name=form.get
('name');let submit=document.getElementById('authSubmit');submit.disabled=true;try{let user=await
apiRequest(`/api/auth/${authMode}`,'POST',payload);setSignedInUser(user);document.getElementById('authForm').reset();await
refreshData(true)}catch(error){showAuth(error.message)}finally{submit.disabled=false}};
0163  document.getElementById('signOutBtn').onclick=async()=>{try{await apiRequest('/api/auth/logout','POST')}finally{showAuth('You have
signed out.');setAuthMode('login')}};
0164  document.getElementById('resetBtn').onclick=async()=>{try{await refreshData(true);toast('Refreshed from
Oracle.')}catch(error){document.getElementById('connectionStatus').innerHTML='<i></i> DB OFFLINE';toast(error.message)}};
0165  async function startApp(){try{let user=await apiRequest('/api/auth/me');setSignedInUser(user);await
refreshData(true)}catch(error){showAuth(error.message==='Please sign in to continue.'?'':'Oracle service is unavailable.
'+error.message)}}
0166  startApp();
0167  setInterval(()=>{if(authUser)refreshData().catch(error=>{if(error.message.includes('sign in'))showAuth('Your session expired. Please
sign in again.');else{document.getElementById('connectionStatus').innerHTML='<i></i> DB
OFFLINE';document.getElementById('connectionStatus').style.color='#bd5d57'}})},5000);
0168  </script>
0169  </body>
0170  </html>
```